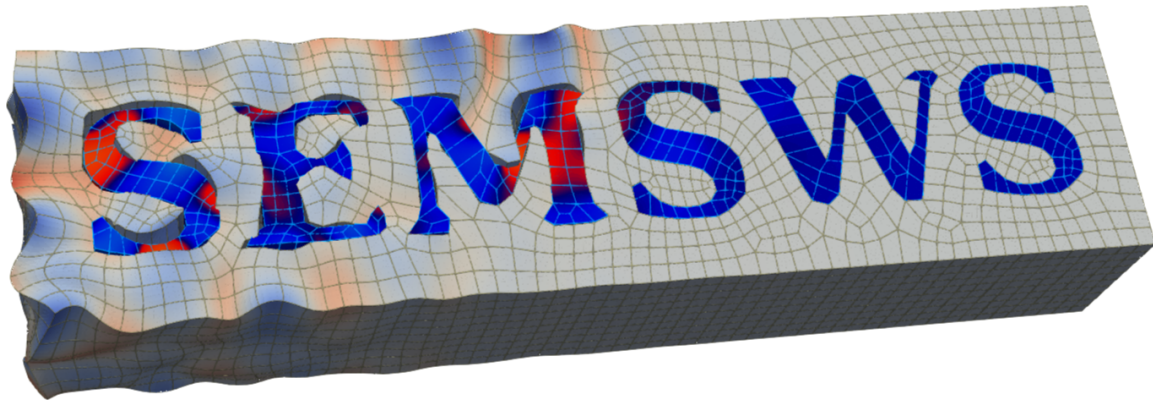




Spectral Element Method
Seismic Wave Simulator

User Manual

Version 0.1

July 2, 2026

Contents

1	Introduction	7
1.1	What is SEMSWS	7
1.2	Supported Physical Models	7
1.3	Structure of This Manual	8
2	Installation	9
2.1	Requirements	10
2.2	Installation with Spack (recommended)	10
2.3	Manual Build	11
2.3.1	hypre	12
2.3.2	MFEM	12
2.3.3	yaml-cpp	13
2.3.4	ADIOS2	13
2.3.5	SEMSWS	13
2.3.6	(Optional) Using Umpire	14
2.4	Verification	15
2.5	GLVis (recommended)	16
3	Quick Start	19
3.1	Configuration File	19
3.1.1	Basic Simulation Settings	19
3.1.2	Mesh	19
3.1.3	Material Properties	20
3.1.4	Boundary Conditions	20
3.1.5	Source	21
3.1.6	Receivers	21
3.1.7	Visualization Output (vis)	21
3.1.8	Device Settings	22
3.2	Running the Simulation	22
3.3	Checking the Results	23
3.3.1	Summary	23
3.3.2	Wavefield Snapshots	24
3.3.3	Receiver Waveforms	24

4	Configuration File Reference	25
4.1	YAML Format Basics	25
4.2	Strict-Key Validation	26
4.3	Overall Structure of the Configuration File	26
4.4	Root-Level Parameters	26
4.5	simulation Section	27
4.5.1	Basic Parameters	27
4.5.2	Time Integration	27
4.5.3	Simulation Mode	27
4.5.4	Output Settings	28
4.6	vis Section	28
4.6.1	Wavefield Output (vis.wavefield)	28
4.6.2	Material Property Output (vis.material)	30
4.6.3	Mesh Visualization (vis.mesh)	30
4.6.4	simulation / vis Configuration Example	30
4.7	mesh Section	31
4.7.1	Common Parameters	31
4.7.2	Internal Mesh (type: internal)	31
4.7.3	External Mesh (type: external)	32
4.7.4	Pre-Partitioned Mesh (type: prepart_mfem)	32
4.7.5	Configuration Examples for the mesh Section	32
4.8	material Section	33
4.8.1	Material Types and Formats	33
4.8.2	format: constant	33
4.8.3	format: grid	34
4.8.4	format: by_attribute	35
4.8.5	format: by_attribute_mixed	35
4.8.6	format: adios2	35
4.8.7	Attenuation	36
4.8.8	type: coupled (Fluid–Solid Coupling)	36
4.8.9	Model Export	38
4.8.10	Configuration Examples for the material Section	38
4.9	boundary Section	39
4.9.1	Absorbing Boundary Condition (absorbing)	39
4.9.2	Dirichlet Boundary Condition (dirichlet)	40
4.9.3	Configuration Example for the boundary Section	40
4.10	sources Section	41
4.10.1	Source Modes and Input Formats	41
4.10.2	Common Source Parameters (list)	41
4.10.3	Source Type: force	41
4.10.4	Source Type: pressure	42
4.10.5	Source Type: moment_tensor	42
4.10.6	Wavelet	42

4.10.7	Observed Data — Inversion Mode	43
4.10.8	Configuration Examples for the sources Section	43
4.11	receivers Section	44
4.11.1	Ways to Specify Receivers	44
4.11.2	Recording Type (type) and Output Settings (output)	44
4.11.3	Receiver List (list)	45
4.11.4	Receiver Line (line)	45
4.11.5	Configuration Examples for the receivers Section	45
4.12	device section	47
4.12.1	Example device section configuration	47
4.13	inversion section	47
4.14	HDF5 I/O Schema and Usage	48
4.14.1	Overview of the HDF5 v2.0 schema (common)	48
4.14.2	Input path: HDF5 → SEMSWS	51
4.14.3	Output path: SEMSWS → HDF5	53
5	Tools and Scripts	57
5.1	partition_mesh — Pre-partitioning a Mesh	57
5.1.1	Usage	57
5.1.2	Options	58
5.1.3	Usage in config.yaml	58
5.2	combine_mesh — Combining Partitioned Meshes	58
5.3	evaluate_mesh — Mesh Quality Evaluation	59
5.3.1	Usage	59
5.3.2	Options	59
5.3.3	Histograms Output	59
5.4	refine_mesh — Uniform Mesh Refinement and Partitioning	59
5.4.1	Mesh Loading and Output Behavior	60
5.4.2	Usage	61
5.4.3	Options	61
5.5	Example Workflow: Mesh Evaluation and Refinement	61
5.5.1	Problem Setup	61
5.5.2	Step 1: Evaluating the Initial Mesh	62
5.5.3	Step 2: Uniform Refinement	63
5.5.4	Step 3: Re-evaluation After Refinement	63
5.5.5	Step 4: Running the Simulation	64
5.6	Visualization Scripts	64
5.6.1	plot_wavefield_2d.py — 2D Wavefield Visualization	64
5.6.2	plot_wavefield_3d.py — 3D Cross-Section Wavefield Visualization	65
5.6.3	plot_3d_slices.py — PyVista 3D Visualization	66
5.6.4	plot_mesh_quality.py — Mesh Quality Histograms	66

A Site-Specific Build Configurations	69
A.1 Earth Simulator (NVIDIA A100 GPU)	69
A.1.1 Manual Build	69
A.1.2 Spack	70
A.1.3 Subsequent Sessions	71
A.1.4 Example PJM Job Script	71
A.2 LUMI (AMD MI250X GPU)	72
A.2.1 Manual Build	72
Development Team	75

Chapter 1

Introduction

1.1 What is SEMSWS

SEMSWS (Spectral Element Method – Seismic Wave Simulator) is a seismic wave propagation simulation solver based on the spectral element method (SEM). It is built on top of the finite element library MFEM¹. MFEM is an open-source library developed primarily at Lawrence Livermore National Laboratory that provides high-order finite element spaces, a variety of mesh formats, MPI-parallel assembly, and GPU/CPU portability via CUDA/HIP/OpenMP. Many of the core features of SEMSWS rely on this MFEM foundation, enabling a single source code to run on both CPUs and GPUs (CUDA/HIP). We gratefully acknowledge the MFEM development team here.

SEM, a widely used numerical method for solving the seismic wave equation, is a technique based on high-order Lagrange interpolation at Gauss–Lobatto–Legendre (GLL) points, and it features efficient explicit time integration through diagonalization of the mass matrix, support for unstructured meshes, natural incorporation of free-surface boundary conditions through the weak form, and highly accurate, low-dispersion solutions based on spectral convergence.

1.2 Supported Physical Models

SEMSWS supports the following wave equations in both 2D and 3D:

- **Isotropic Elastic** wave equation
- **Isotropic Acoustic** wave equation
- **Viscoelastic / Viscoacoustic**
- **Fluid–Solid Coupling**

Support for additional models such as VTI and Full Anisotropy is planned for future updates.

¹<https://mfem.org/>

1.3 Structure of This Manual

Chapter 2: Installation Explains the procedure from preparing the dependent libraries (hypre, MFEM) to building the code. Also includes example build configurations for each computing environment (local, ES, LUMI).

Chapter 3: Quick Start Using a 2D acoustic wave simulation as an example, explains the full workflow from writing the configuration file through execution to visualizing the results.

Chapter 4: Configuration File Reference Comprehensively describes all options of `config.yaml`.

Chapter 5: Tools and Scripts Explains how to use the mesh partitioning, refinement, and quality-evaluation tools, as well as the visualization scripts.

Chapter 2

Installation

This chapter describes two ways to build SEMSWS itself.

- [§2.2](#): Automatic installation with Spack (recommended)
- [§2.3](#): Manual build (building the dependency libraries yourself)

The list of dependency libraries is summarized in [§2.1](#), and post-build verification is covered in [§2.4](#). For HPC-site-specific settings such as Earth Simulator or LUMI, see Appendix [A](#).

2.1 Requirements

Table 2.1: Dependency libraries

Software	Required/Optional	Notes
C++17 compiler	Required	Must support <code>-std=c++17</code>
CMake ≥ 3.20	Required	Build system
MPI	Required	Parallel computation
MFEM ≥ 4.9	Required	Must be built with MPI enabled. Depends on the <code>mfem::Reshape</code> and <code>Dual<></code> automatic-differentiation types introduced in 4.9
hypre	Required	Linked via MFEM
ADIOS2	Required	Parallel I/O (BP format)
HDF5	Required	Waveform and receiver I/O. Linked directly by SEM-SWS. Must be the same HDF5 as used by MFEM (specify <code>HDF5_ROOT</code> if not auto-detected)
yaml-cpp $\geq 0.9.0$	Required	Configuration file parsing. 0.7/0.8 lack the <code><csdint></code> fix and cannot be built (not fetched automatically, so prepare it in advance)
METIS	Required	Mesh partitioning
NetCDF	Optional	Reading Cubit/Exodus meshes. Linked via MFEM (<code>MFEM_USE_NETCDF</code>)
CUDA	Optional	NVIDIA GPU support
HIP	Optional	AMD GPU support
Umpire	Optional	GPU memory pool. Performance impact is small and it is not required (§2.3.6)

Python packages for pytest To run the test suite in `test/`, install the following in addition to Python 3.10 or later:

```
pip install pytest numpy h5py pyyaml matplotlib gmsh
```

`matplotlib` and `gmsh` are optional (used for plotting and fluid–solid coupling mesh-generation tests, respectively). For the Python driver `semsws-driver` under `driver/`, dependencies are resolved automatically from `driver/pyproject.toml` via `pip install -e ./driver`.

2.2 Installation with Spack (recommended)

The SEMSWs repository ships with a Spack recipe under `spack-repo/`, which can automatically build all dependency libraries, including hypre, MFEM, and ADIOS2. Spack 1.0 or later is required (verified with v1.1.x and v1.2.x).

Step 1: Clone SEMSWs

```
git clone https://github.com/SEMSWS/semsws.git
cd semsws
```

Step 2: Prepare Spack If the `spack` command is not available, do the following:

```
git clone --depth=1 https://github.com/spack/spack.git ~/spack
source ~/spack/share/spack/setup-env.sh
```

The `source` line is needed every time you open a new shell, so it is convenient to add it to `~/.bashrc`.

Step 3: Register the semsws Spack repo

```
spack repo add ./spack-repo
```

Step 4: Register cmake as external (recommended) Without this step, Spack builds `cmake` from source, which takes more than 10 minutes, so it is best to declare the system `cmake` (3.20 or later) as external:

```
mkdir -p ~/.spack
cat >> ~/.spack/packages.yaml <<'EOF'
packages:
  cmake:
    externals:
      - spec: cmake@3.28.0      # match `cmake --version`
        prefix: /usr           # prefix as confirmed with `which cmake`
        buildable: false
EOF
```

Step 5: Install and load

```
spack install semsws@main
spack load semsws
which semsws      # /path/to/spack/opt/.../semsws-.../bin/semsws
```

The initial build, including dependency libraries, takes 30–60 minutes.

Site-specific environments (ES, LUMI) Earth Simulator and LUMI require site-specific adjustments such as compiler patches, precision settings, and GPU-aware MPI. Loader scripts and `packages.yaml` templates are provided under `spack-repo/site-config/<site>/`. See Appendix A for concrete specs and procedures.

2.3 Manual Build

This section describes the procedure for building the dependency libraries yourself without Spack. SEMSWs requires `hypr`, `MFEM`, `yaml-cpp`, `ADIOS2`, and `HDF5` as mandatory dependencies, none of which are fetched automatically, so they must be built in advance (`hypr` must precede `MFEM` because their GPU precision settings must match; `yaml-cpp`, `ADIOS2`, and `HDF5` are independent and may be built in any order). Finally, build SEMSWs itself. Umpire is not required; add the steps in §2.3.6 only if you choose to use it.

Note: the commands in this section are only examples. Paths, compilers, GPU architectures, module configurations, and library versions differ from environment to environment, so adapt them to your own environment.

_DIR versus _ROOT When passing the location of a dependency library to CMake, `-DXxx_DIR` points to the directory containing `XxxConfig.cmake` (the CMake config directory), while `-DXxx_ROOT` points to the install prefix. If a dependency is already installed, using `_ROOT` is simpler. **Exception:** when pointing to hypre during the MFEM build itself, `HYPRE_ROOT` has no effect because MFEM uses its own find script; pass hypre’s install directory via `HYPRE_DIR` instead.

2.3.1 hypre

Build hypre with CMake under `src/`. The minimal CPU (single-precision) configuration is:

```
cd hypre/src
cmake -B build \
  -DHYPRE_ENABLE_SINGLE=ON -DHYPRE_ENABLE_MPI=ON \
  -DCMAKE_INSTALL_PREFIX=$(pwd)/install
cmake --build build -j$(nproc)
cmake --install build
```

For double precision, omit `HYPRE_ENABLE_SINGLE=ON`. For GPU builds, add the flags in Table 2.2 depending on the backend.

Table 2.2: hypre CMake flags (differences by backend)

Backend	Additional flags
CPU	(none)
CUDA	<code>-DHYPRE_ENABLE_MIXEDINT=ON -DHYPRE_ENABLE_UNIFIED_MEMORY=ON</code> <code>-DHYPRE_ENABLE_CUDA=ON -DHYPRE_ENABLE_GPU_AWARE_MPI=ON -</code> <code>DCMAKE_CUDA_ARCHITECTURES=80 -DHYPRE_ENABLE_CUSOLVER=OFF</code>
HIP	<code>-DHYPRE_ENABLE_MIXEDINT=ON -DHYPRE_ENABLE_UNIFIED_MEMORY=ON</code> <code>-DHYPRE_ENABLE_HIP=ON -DHYPRE_ENABLE_GPU_AWARE_MPI=ON -</code> <code>DCMAKE_HIP_ARCHITECTURES=gfx90a</code>

`HYPRE_ENABLE_CUSOLVER` is not needed by SEMSW, so it can be left OFF. Linking cuSOLVER causes `extern "C"`-related compile errors on older GCC (e.g., the 8.x series), so disabling it avoids trouble. For SEMSW, hypre 2.33.0 is the most stable version. Building GPU support without Umpire may cause hypre to emit warnings, which can be safely ignored. (For using Umpire together with hypre, see §2.3.6.)

2.3.2 MFEM

SEMSWS requires MFEM 4.9 or later. An example CMake build is shown below:

```
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=mpicxx \
  -DMFEM_USE_MPI=ON \
  -DMFEM_USE_HDF5=ON \
  -DMFEM_USE_NETCDF=ON \
```

```
-DMFEM_PRECISION=single \
-DHYPRE_DIR=/path/to/hypre/install
cmake --build build -j$(nproc)
cmake --install build
```

Pass hypre's install directory via `HYPRE_DIR` (the exception to the convention noted above). NetCDF is not required, but is needed to read meshes generated by tools such as CUBIT. For GPU builds, switch to `-DMFEM_USE_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=80` (NVIDIA) or `-DMFEM_USE_HIP=ON -DAMDGPU_TARGETS=gfx90a` (AMD), along with the corresponding `CMAKE_CXX_COMPILER`. On clusters with multiple GPU device types, `CMAKE_CUDA_ARCHITECTURES` may not work correctly; in that case, specify it explicitly with `-DCUDA_ARCH=sm_80`.

Precision consistency hypre's `HYPRE_ENABLE_SINGLE` and MFEM's `MFEM_PRECISION=single` must always match. SEMSWs automatically inherits MFEM's precision setting, so as long as hypre and MFEM agree, no re-specification is needed on the SEMSWs side.

2.3.3 yaml-cpp

Used to parse configuration files. Version 0.9.0 or later is required (0.7/0.8 will not work). Fetch and build the corresponding tag:

```
git clone -b 0.9.0 https://github.com/jbeder/yaml-cpp.git
cd yaml-cpp
cmake -B build -DYAML_BUILD_SHARED_LIBS=ON \
  -DCMAKE_INSTALL_PREFIX=$(pwd)/install
cmake --build build -j$(nproc)
cmake --install build
```

2.3.4 ADIOS2

Used for FWI kernel/model I/O (BP5 format). Build with MPI enabled:

```
cmake -B build -DADIOS2_USE_MPI=ON \
  -DADIOS2_USE_Python=OFF -DBUILD_TESTING=OFF \
  -DCMAKE_INSTALL_PREFIX=$(pwd)/install
cmake --build build -j$(nproc)
cmake --install build
```

2.3.5 SEMSWs

Once all dependency libraries are installed, build SEMSWs itself, passing their locations via `_ROOT` following the convention described earlier:

```
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DMFEM_ROOT=/path/to/mfem/install \
  -Dyaml-cpp_ROOT=/path/to/yaml-cpp/install \
  -DAdios2_ROOT=/path/to/ADIOS2/install \
```

```

-DCMAKE_CUDA_ARCHITECTURES=80 \
-DCMAKE_INSTALL_PREFIX=$(pwd)/install
cmake --build build -j$(nproc)
cmake --install build

```

hypre, METIS, and NetCDF are linked transitively via MFEM, so they do not need to be specified on the SEMSWs side. Only HDF5 is looked up directly by SEMSWs via `find_package`, but the system-installed HDF5 is normally detected automatically. If it is not found, pass the same HDF5 prefix used by MFEM via `-DHDF5_ROOT=`.

Table 2.3: Key CMake options

Option	Description
MFEM_DIR / MFEM_ROOT	Location of MFEM (required). <code>_DIR</code> is the CMake config directory, <code>_ROOT</code> is the install prefix (per the convention above)
adios2_DIR / adios2_ROOT	Location of ADIOS2 (same as above)
CMAKE_BUILD_TYPE	Release (recommended) or Debug
CMAKE_INSTALL_PREFIX	Destination for <code>cmake --install</code>
CMAKE_CUDA_ARCHITECTURES	For NVIDIA GPUs (e.g., 80)
AMDGPU_TARGETS	For AMD GPUs (e.g., <code>gfx90a</code>)
SEM_FORCE_CPU	Build CPU-only even if MFEM supports GPU (default OFF)
SEM_USE_GPU_AWARE_MPI	Assert at build time that GPU-aware MPI is available (default OFF)
SEM_ENABLE_PROFILING	Enable per-region timing measurement (incurs MFEM_DEVICE_SYNC overhead; default OFF)

Specify `SEM_USE_GPU_AWARE_MPI=ON` only in environments confirmed to be GPU-aware, such as LUMI Cray MPICH (`MPICH_GPU_SUPPORT_ENABLED=1`) or CUDA-aware OpenMPI.

If MFEM was built with CUDA or HIP support, SEMSWs automatically becomes a GPU build. To force a CPU-only build, specify `SEM_FORCE_CPU=ON`.

2.3.6 (Optional) Using Umpire

Umpire is a library that manages memory pools on the GPU, but its performance impact on SEMSWs is small and it is not required. It is unnecessary for CPU builds, and may be omitted even for GPU builds. Add the following only if you want to explicitly suppress memory fragmentation with a pool.

First, build Umpire:

```

cmake -B build \
-DENABLE_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=80 \
-DENABLE_MPI=ON -DUMPIRE_ENABLE_C=ON \
-DCMAKE_INSTALL_PREFIX=$(pwd)/install
cmake --build build -j$(nproc)
cmake --install build

```

Then add the following to the `cmake` invocation of each library:

- hybre: `-DHYPRE_ENABLE_UMPIRE=ON -Dumpire_ROOT=/path/to/umpire/install`
- MFEM: `-DMFEM_USE_UMPIRE=ON -DUMPIRE_ROOT=/path/to/umpire/install`
- SEMSW: `-Dumpire_ROOT=/path/to/umpire/install`

If hybre and MFEM are built with Umpire, be sure to also build SEMSW with Umpire so that the configuration remains consistent.

2.4 Verification

A successful build produces a set of executables under `build/src/`. The core components are the simulation executable and the mesh/visualization tools listed in Table 2.4.

Table 2.4: Key executables produced

Executable	Description
<code>semsws</code>	Simulation executable
<code>partition_mesh</code>	Mesh pre-partitioning
<code>refine_mesh</code>	Mesh refinement
<code>evaluate_mesh</code>	Mesh quality evaluation
<code>combine_mesh</code>	Combining partitioned meshes (for visualization)
<code>sem_viz</code>	Converts ADIOS2 BP (material/kernel fields) to GLVis/ParaView/GMT format

In addition to the above, a set of FWI kernel-processing and model-update tools (`smooth_kernel`, `sum_kernels`, `mask`, `precondition_gradient`, `clip_field`, `scale_field`, `lbfgs_direction`, `update_model`, `max_abs`, `semsws_export_model`) is also produced. Their usage is covered in Chapter 5.

```
./build/src/semsws --help
```

Verification with pytest `test/` contains tests organized by purpose. Pass the location of the executables via `--build-dir`: for a local build this is the build directory (`./build`); for installations via `cmake --install` or `Spack`, specify the install prefix.

```
# All tests (CPU, serial)
pytest test/ --build-dir ./build -v

# MPI parallel, with GPU device specified
pytest test/ --build-dir ./build --np 4 --device cuda -v

# SLURM cluster (replace launcher with srun; --build-dir is the install
  location)
pytest test/ --build-dir <install_dir> --device cuda \
  --mpi-cmd=srun --np 2 -s -v -x
```

```
# Only waveform verification, output-format consistency, or fluid--solid
  coupling
pytest test/waveform/      --build-dir ./build -v
pytest test/consistency/  --build-dir ./build -v
pytest test/coupling/     --build-dir ./build --np 4 -v
```

The **waveform verification tests** (`test/waveform/`) compare simulation results against reference waveforms and verify that the normalized L2 norm is within a threshold. Cases whose names include **analytic** are compared against analytical solutions; all others are compared against SPECSEM. With `--keep-results`, simulation results are saved to **results/** in each test directory, allowing direct comparison with **ref/** in the same directory.

The **output-format consistency tests** (`test/consistency/`) verify that waveforms match numerically across receiver output formats (ASCII, HDF5, SU). The **coupling tests** (`test/coupling/`) verify fluid-solid coupling problems.

For a Spack installation, pass the prefix obtained from `spack find --paths semsws` via `--build-dir`:

```
SEMSWS_PREFIX=$(spack find --paths semsws | awk 'NR>1 && /\/// {print $NF;
  exit}')
pytest test/consistency/ --build-dir $SEMSWS_PREFIX --np 4 -v
```

2.5 GLVis (recommended)

GLVis is a lightweight visualization tool published by the MFEM project¹. It can read the MFEM-format meshes and grid functions that SEMSWS writes to **results/wavefield/**, allowing the wavefield to be inspected frame by frame. It is not required for SEMSWS itself to run, but it makes the Quick Start tutorial in Chapter 3 much easier to verify. (To visualize material fields or FWI kernels output in ADIOS2 BP format, use `sem_viz` from Table 2.4 to convert them to GLVis/ParaView/GMT format. See Chapter 5 for details.)

Build GLVis can reference the MFEM installed in the previous section directly. On Debian / Ubuntu, install the system packages beforehand:

```
sudo apt install libsdl2-dev libglew-dev libfontconfig-dev libfreetype-dev

git clone https://github.com/GLVis/glvis.git
cd glvis
make -j MFEM_DIR=../mfem
```

Basic usage

```
# Mesh only
glvis -m mesh.mesh

# Mesh + one frame of the wavefield
glvis -m results/wavefield/mesh.mesh -g results/wavefield/sol_000500.gf
```

¹<https://glvis.org/>

```
# Partitioned (MPI parallel) mesh -- N is the number of output ranks  
glvis -np 2 -m results/wavefield/mesh -g results/wavefield/sol_000500
```

For partitioned mesh formats, GLVis automatically reads `mesh.000000`, `mesh.000001`, ...and the corresponding `sol_000500.000000...`, recombining them into a single window for display. For key bindings, see the GLVis README².

²<https://github.com/GLVis/glvis>

Chapter 3

Quick Start

This chapter walks through the complete workflow, from writing a configuration file to running the simulation and visualizing the results, using a 2D acoustic wave simulation as an example. The example used here is located in `examples/visualization/2D/acoustic/`.

3.1 Configuration File

SEMSWS simulations describe all parameters in a single YAML file (`config.yaml`). The following sections walk through the configuration file for this example, section by section.

3.1.1 Basic Simulation Settings

```
name: Visualization Test - 2D Acoustic

simulation:
  dimension: 2
  order: 4
  steps: 10000
  dt: 0.0001
  cfl_factor: 0.3
  directory: ./results
  log_interval: 1000
  summary_file: summary.txt
```

`dimension` specifies the spatial dimension (2 or 3), and `order` specifies the polynomial order (the order of the spectral element method). `steps` and `dt` set the number of time steps and the time step size Δt . In this example, $\Delta t = 0.0001$ s with 10000 steps, giving a simulation duration of 1 second. `cfl_factor` is the safety factor for the CFL stability condition, `directory` is the output directory, `log_interval` is the interval for progress logging, and `summary_file` is the name of the summary output file. The `name` field at the top is an identifier for the simulation (optional).

3.1.2 Mesh

```

mesh:
  type: internal
  origin: [0.0, 0.0]
  size: [1000.0, 500.0]
  elements: [50, 25]
  max_freq: 75.0
  ppw: 5.0

```

`type: internal` tells SEMSWS to automatically generate a regular/structured mesh internally. `size` specifies the domain size (m), and `elements` specifies the number of elements. In this example, a 1000 m $\times$ 500 m domain is divided into 50 $\times$ 25 elements. Mesh visualization output is enabled via `vis: mesh: enabled: true`, described later. `max_freq` and `ppw` are parameters used for mesh quality evaluation, representing the maximum frequency and the number of sampling points per wavelength (points per wavelength), respectively. These are used in the `summary.txt` output and in the `evaluate_mesh` program described later.

To use an external mesh (GMSH, CUBIT/ExodusII, etc.), set `type: external` and specify the file path in `file::`

```

mesh:
  type: external
  file: "model.exo"

```

3.1.3 Material Properties

```

material:
  type: isotropic_acoustic
  format: constant
  vp: 1500.0
  rho: 1000.0
  attenuation:
    enabled: false

```

This defines a homogeneous acoustic medium. `type` selects the physical model (`isotropic_acoustic`), and `format: constant` specifies that this is a homogeneous model. The settings give a P-wave velocity of 1500 m/s and a density of 1000 kg/m³.

3.1.4 Boundary Conditions

```

boundary:
  absorbing:
    sides: []
    thickness: 100.0
    alpha: 0.008

  dirichlet: [top, right, left, bottom]

```

`dirichlet` sets a Dirichlet (fixed) boundary condition on all sides. The absorbing boundary (Cerjan sponge layer) is disabled via `sides: []`. To enable the absorbing boundary, specify the sides it applies to, e.g. `sides: [top, right, left, bottom]`. For simulations of marine reflection or refraction seismic surveys, **the sea surface must be set to Dirichlet** (`dirichlet: [top]`).

3.1.5 Source

```
sources:
  shot_id: 1
  list:
    - type: pressure
      location: [500.0, 400.0]
      wavelet:
        type: ricker
        frequency: 30.0
        amplitude: 1.0e10
        delay: 0.05
```

A single pressure point source with a 30 Hz Ricker wavelet is placed near the center of the domain (500 m, 400 m). `delay` is the time (in seconds) at which the wavelet reaches its peak. `shot_id` is the ID of this shot. Each source may optionally be given an `id` and `name`.

3.1.6 Receivers

```
receivers:
  output:
    formats: [ascii]
    filename: seismograms
    type: [PS]

  list:
    - name: "R001"
      location: [400.0, 300.0]
    - name: "R002"
      location: [500.0, 300.0]
    - name: "R003"
      location: [600.0, 300.0]
```

Three receivers are placed to record the pressure scalar field (PS). `formats` is a list of output formats; `ascii`, `hdf5`, and `su` (Seismic Unix format) can be specified. With `ascii`, a text file is produced whose first column is time and second column is amplitude. `filename` is the base name for HDF5/SU output; ASCII output is named per receiver, e.g. `R001_0001.p`.

3.1.7 Visualization Output (vis)

Visualization output for the wavefield, material properties, and mesh is configured together under the top-level `vis:` section.

```
vis:
  wavefield:
    enabled: true
    interval: 1000
    fields: ["PS"]
    components: [-1]
    formats:
      - glvis
      - type: paraview
        refinement: 4
        data_format: binary32
      - type: gmt
        resolution: [100, 50]
  material:
    enabled: true
    fields: [vp, rho]
    formats:
      - glvis
  mesh:
    enabled: true
```

`vis: wavefield` outputs a wavefield snapshot every 1000 steps. `formats` can specify multiple formats simultaneously: `glvis`, which requires no options, is given as a plain string, while ParaView and GMT pass additional settings via a map with `type::`

- **GLVis** — MFEM’s native format. Can be visualized interactively with the `glvis` command
- **ParaView** — VTU format. Well suited to 3D visualization in ParaView
- **GMT** — Text grid format (`x y value`). `resolution` specifies the grid resolution

`vis: material` outputs the distribution of material properties (`vp`, `rho`, etc.), and `vis: mesh: enabled: true` outputs the mesh. `components: [-1]` represents all components of a vector field (ignored for scalar fields).

3.1.8 Device Settings

```
device:
  type: cpu
```

Choose from `cpu`, `cuda`, or `hip`. Even in a GPU build, specifying `cpu` will run the simulation on the CPU (however, this is considerably slower, so a CPU build is recommended when running on CPU).

3.2 Running the Simulation

Run the following command in the directory containing the configuration file:

```
cd examples/visualization/2D/acoustic

# Run with 4 processes
mpirun -np 4 ../../../../build/src/semsws \
  --config config.yaml
```

You can also use `run.sh`, included in the same directory:

```
bash run.sh 4    # argument is the number of MPI processes
```

Once the run completes, the following files are generated in the `results/` directory:

Table 3.1: Output file listing

Path	Description
<code>summary.txt</code>	Simulation information (material properties, mesh, performance, etc.)
<code>R001_0001.p</code> , etc.	Receiver waveforms (ASCII text)
<code>wavefield/gmt/</code>	GMT snapshots
<code>wavefield/paraview/</code>	VTU files for ParaView
<code>wavefield/glvis/</code>	Binary files for GLVis
<code>material/</code>	Material property distribution output
<code>mesh.mesh</code>	The mesh used

3.3 Checking the Results

3.3.1 Summary

A summary of the simulation is recorded in `results/summary.txt`. An example of the output for this example is shown below:

```
Discretization:
  Order: 4
  Elements: 1250
  DOFs: 20301

Time stepping:
  dt: 0.0001 s
  nt: 10000
  Simulation time: 1 s

Numerical stability:
  h_min: 20.00 m, h_max: 20.00 m
  CFL dt_max: 7.3535e-04 s (dt/dt_max = 0.14)
  PPW: 12.5 (f=30.0 Hz)

Performance:
  Setup time: 0.18 s
```

```
Run time:      2.27 s
Total time:    2.44 s
DOFs/sec:     8.96e+07
```

In particular, verify that CFL $\Delta t_{\max}$ is larger than the configured Δt . If $\Delta t > \Delta t_{\max}$, the computation becomes unstable and diverges. PPW (Points Per Wavelength) is the number of grid points per wavelength; a value of 5 or greater is a common guideline.

3.3.2 Wavefield Snapshots

The wavefield can be visualized using the included Python script:

```
python3 scripts/plot/plot_wavefield_2d.py --results-dir ./results
```

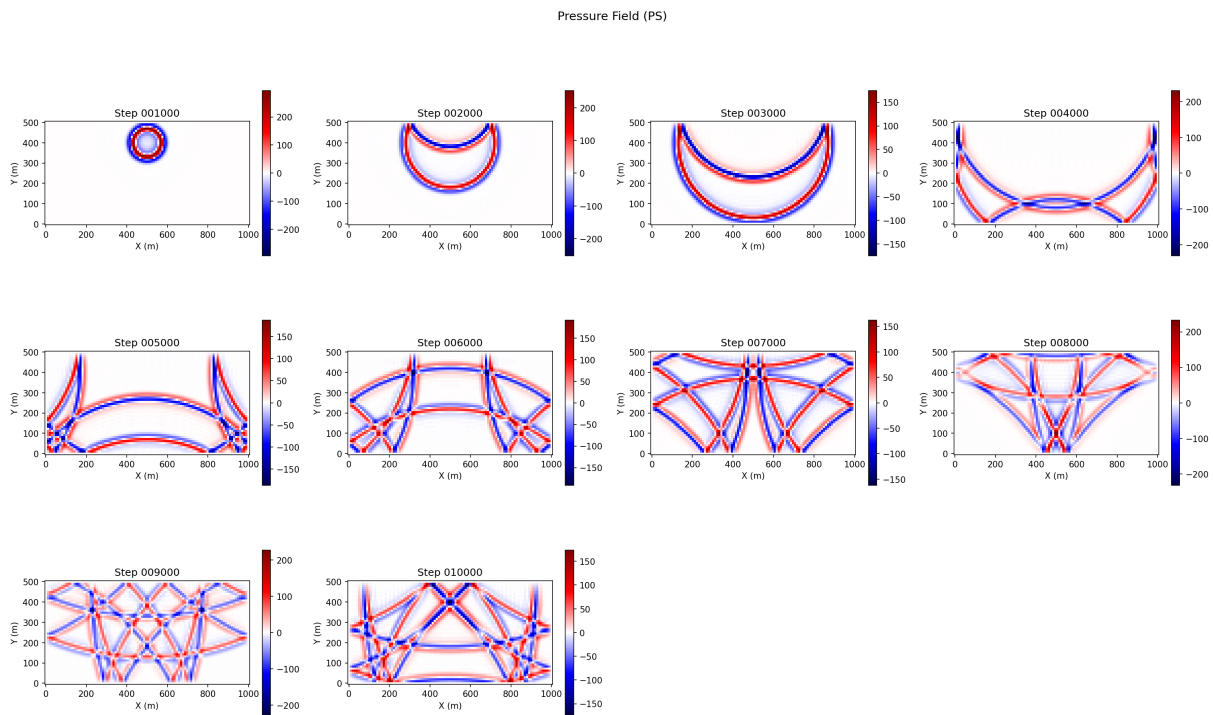


Figure 3.1: Pressure field snapshots (1000-step interval).

Since this example sets a Dirichlet boundary condition on all sides, you can observe the wave reflecting off the boundary surfaces. In actual seismic wave simulations, an absorbing boundary condition is used to suppress such artificial reflections.

3.3.3 Receiver Waveforms

Receiver waveform files are in a two-column ASCII text format. The first column is time (seconds) and the second column is amplitude (pressure). These can easily be plotted using gnuplot, Python, or similar tools.

Chapter 4

Configuration File Reference

SEMSWS simulations specify all parameters in a single YAML file (`config.yaml`). This chapter comprehensively documents every configuration file option. For basic usage, see Chapter [3](#).

4.1 YAML Format Basics

The configuration file is written in YAML (YAML Ain't Markup Language) format. The basic YAML syntax rules are shown below.

- Indentation represents the hierarchical structure. Use **spaces only** for indentation (tabs are not allowed), and keep the indentation width consistent within the same level.
- Write keys and values in the form `key: value` (a space is required after the colon).
- Lists can be written in block form (each element starting with `-`) or flow form (`[a, b, c]`). Maps can also use the flow form `{key: value}`.
- Everything after `#` to the end of the line is ignored as a comment.
- Strings can be written without quotes, but if they contain special characters, or are words that could be interpreted as booleans such as `yes/no/on/off`, they must be enclosed in `"..."` or `'...'`.

```
# Scalar values
name: "My Simulation"
order: 4
dt: 0.0001

# Nested structure
simulation:
  dimension: 2
  steps: 10000
  dt: 0.0001

# List
sides: [bottom, right, top, left]
```

```
# List (multi-line form)
list:
  - name: "R001"
    location: [400.0, 300.0]
  - name: "R002"
    location: [500.0, 300.0]
```

4.2 Strict-Key Validation

The YAML loader checks each block of the configuration file against a set of known keys. If an unknown key appears anywhere (a typo, a renamed old key, a sub-block copy-pasted from an old configuration file, etc.), it is not silently ignored; instead the run aborts with a runtime error in the following form:

```
Error validating config: Unknown key `<bad_key>` in <section>
  (allowed: <list of keys valid in that block>)
```

4.3 Overall Structure of the Configuration File

The overall structure of the configuration file is as follows:

```
name: "... "           # Simulation name (optional)

simulation:             # Basic simulation settings / output
mesh:                  # Mesh
material:               # Material properties
boundary:              # Boundary conditions
sources:               # Sources
receivers:             # Receivers
device:                # Execution device
inversion:             # Inversion settings (optional)
vis:                   # Visualization / wavefield / material / mesh
  output (optional)
```

4.4 Root-Level Parameters

Key	Type	Required	Default	Description
name	string	optional	"Seismic wave simulation"	Simulation name. Recorded in <code>summary.txt</code>

4.5 simulation Section

4.5.1 Basic Parameters

Key	Type	Required	Value	Description
<code>dimension</code>	int	required	2, 3	Spatial dimension
<code>order</code>	int	required	≥ 1	Polynomial order (number of GLL points = $N + 1$)

In the GPU implementation there is an upper bound on `order` due to shared memory constraints.

4.5.2 Time Integration

Key	Type	Required	Default	Description
<code>dt</code>	real	required	—	Time step size (s)
<code>steps</code>	int	required	—	Number of time steps
<code>cfl_factor</code>	real	optional	0.3	CFL safety factor (positive real number; 0.7 or less recommended)
<code>t0</code>	real	optional	0.0	Simulation start time (s)

Δt must satisfy the CFL condition ($\Delta t < \Delta t_{\max}$). `cfl_factor` is used for mesh quality evaluation. The total simulation time is determined by `steps` $\times$ Δt .

4.5.3 Simulation Mode

Key	Type	Required	Default	Description
<code>mode</code>	string	optional	<code>forward</code>	<code>forward</code> , <code>inversion</code> , <code>misfit_only</code>

FWI-related parameters (`misfit_type`, checkpoint settings, kernel output destination, etc.) are specified not in `simulation` but in the top-level `inversion` section. `inversion` is only referenced when `mode` is `inversion` or `misfit_only`. The allowed keys are `misfit_type` (`l2_waveform` / `normalized_correlation`), `kernel_dir`, `sensitivity_backend`, and `checkpoints` (sub-keys `n` / `storage` (`memory` / `disk`) / `device` (`auto` / `host` / `device`) / `dir` (required when `storage: disk`)).

`forward` is the standard forward simulation. `inversion` and `misfit_only` are for Full Waveform Inversion (FWI). `inversion` performs gradient computation via the checkpointing algorithm and adjoint method, while `misfit_only` computes the waveform misfit. Currently only 2D acoustic waves are supported. The other keys are also for FWI and have not yet been thoroughly debugged.

4.5.4 Output Settings

Key	Type	Required	Default	Description
<code>directory</code>	string	required	—	Output root directory
<code>summary_file</code>	string	optional	""	Summary file name (disabled with an empty string)
<code>log_interval</code>	int	optional	99999	Log output interval (in steps)

All output files are written to the directory specified by `directory`. A summary of the computation is written to `summary_file` (specifying something like `summary.txt` is recommended; specifying an empty string disables output). `log_interval` specifies how often (in steps) the progress of the computation is printed. Each time progress is printed, MPI communication occurs in parallel runs, and in particular Host–Device communication occurs in GPU runs, so the interval should be set to an appropriate value.

4.6 vis Section

Visualization output for the wavefield, material properties, and mesh is specified in the top-level `vis` section (outside of `simulation`). The allowed keys are `directory` / `wavefield` / `material` / `mesh`.

4.6.1 Wavefield Output (`vis.wavefield`)

Key	Type	Required	Default	Description
<code>enabled</code>	bool	optional	false	Enable wavefield snapshot output
<code>interval</code>	int	conditional	—	Output interval (in steps; required when enabled)
<code>fields</code>	string array	optional	["DISP"]	Fields to output
<code>formats</code>	array	conditional	—	List of output formats (required when enabled)
<code>components</code>	int array	optional	[-1]	Component specification for GMT output (applied uniformly to all formats, used only by GMT)

Each element of `formats` can be written in one of two ways.

- **Write just the format name** (when no additional options are needed). Example: `glvis`, `paraview`.
- **Write it as a map with a type key** (when adding options). Specify the format name in `type` and list additional settings alongside it in the same element.

```

formats:
  - glvis                                # Name only (no options)
  - type: paraview                        # Map form (type + options)
    refinement: 2
  - type: gmt                            # gmt must always be written in map form
    resolution: [200, 200]

```

Both `glvis` and `paraview` can be written either way. `gmt`, on the other hand, cannot be specified by name alone and must always be written as a map with a `type` key (since it has required options `resolution`, and in 3D, `cross_sections`).

Values that can be specified in `fields`: `DISP` (displacement), `VEL` (velocity), `ACC` (acceleration), `PS` (pressure scalar, only for acoustic wave simulations).

Output formats Specify the following options for each element of the `formats` array:

Key	Applies to	Default	Description
<code>type</code>	all	—	<code>glvis</code> , <code>paraview</code> , <code>gmt</code>
<code>refinement</code>	<code>paraview</code>	1	Subdivision level
<code>data_format</code>	<code>paraview</code>	<code>binary32</code>	<code>ascii</code> , <code>binary</code> (64-bit), <code>binary32</code> (32-bit)
<code>compression</code>	<code>paraview</code>	−1	zlib compression level (0–9, −1 = default; likely requires MFEM built with zlib, untested)
<code>resolution</code>	<code>gmt</code>	—	Number of output grid points (resolution) [<code>nx</code> , <code>ny</code>]
<code>cross_sections</code>	<code>gmt</code> (3D)	—	Cross-section positions

For 3D GMT output, cross-section positions are specified with `cross_sections`:

```

cross_sections:
  yz: [2500.0]      # yz cross-section at x=2500m
  xz: [2500.0]      # xz cross-section at y=2500m
  xy: [1000.0]      # xy cross-section at z=1000m

```

Per-side overrides (for fluid–solid coupling). When using `material.type: coupled`, the fluid side and the solid side are defined on separate `ParSubMesh` instances, and wavefield output files are also written out separately per side. By adding `fluid:` or `solid:` sub-blocks inside the `vis.wavefield:` block, you can override `enabled` / `interval` / `fields` / `formats` / `components` for that side only. Any key not specified is inherited from the parent `wavefield:` settings. Setting `enabled: false` on one side stops output for that side only, while the other side continues to be written out normally.

```

wavefield:
  enabled: true
  interval: 500
  formats:
    - type: glvis

```

```

fluid:
  fields: [PS]           # Fluid side: pressure only
solid:
  fields: [DISP, VEL]    # Solid side: displacement + velocity

```

4.6.2 Material Property Output (vis.material)

Key	Type	Required	Default	Description
enabled	bool	optional	false	Enable material property output
fields	string array	optional	["vp"]	Fields to output
formats	array	conditional	—	Formats (same as wavefield)

Values that can be specified in **fields**: **vp**, **vs**, **rho**, **qkappa**, **qmu**. Material property output is written only once, at the start of the simulation.

For simulations with **material.type**: **coupled**, as with the wavefield above, **fluid** / **solid**: sub-blocks can also be placed under **vis.material** to override **enabled** / **fields** / **formats** per side (e.g., outputting **vs** only for the solid side).

4.6.3 Mesh Visualization (vis.mesh)

The only key that **vis.mesh** has is **enabled**; setting **enabled**: **true** enables visualization output of the mesh (showing the partitioning).

```

vis:
  mesh:
    enabled: true

```

4.6.4 simulation / vis Configuration Example

```

simulation:
  dimension: 3
  order: 4
  steps: 20000
  dt: 0.00005
  cfl_factor: 0.3
  directory: "./results"
  summary_file: "summary.txt"
  log_interval: 500

vis:
  wavefield:
    enabled: true
    interval: 1000
    fields: ["DISP"]
    components: [-1, 0, 1, 2]

```

```

formats:
  - type: "paraview"
    refinement: 2
    data_format: "binary32"
  - type: "gmt"
    resolution: [200, 200]
    cross_sections:
      xy: [0.0]
      xz: [2500.0]

material:
  enabled: true
  fields: ["vp", "vs", "rho"]
  formats:
    - type: "paraview"
      refinement: 2

```

4.7 mesh Section

4.7.1 Common Parameters

Key	Type	Required	Default	Description
type	string	required	—	internal, external, prepart_mfem
max_freq	real	required	—	Maximum frequency for mesh quality evaluation (Hz)
ppw	real	required	—	Points Per Wavelength

4.7.2 Internal Mesh (type: internal)

Automatically generates a structured-grid mesh inside SEMSWs.

Key	Type	Required	Default	Description
origin	real array	required	—	Origin coordinates [x,y] or [x,y,z] (m)
size	real array	required	—	Domain size [Lx,Ly] or [Lx,Ly,Lz] (m)
elements	int array	required	—	Number of elements [Nx,Ny] or [Nx,Ny,Nz]
partition_method	string	optional	metis	metis or cartesian
partition_grid	int array	conditional	—	Partition counts [nx,ny,nz] when cartesian

partition_method: cartesian is a structured partitioning that does not require METIS. For partition_grid, specifying, e.g., [4, 4, 2] in three dimensions partitions the domain into 4,

4, and 2 parts along the x, y, and z axes respectively. In this case, the degree of parallelism in the run command (e.g., `-np 4`) must match the product of the partition counts, i.e., 32.

4.7.3 External Mesh (type: external)

Key	Type	Required	Default	Description
<code>file</code>	string	required	—	Mesh file path (MFEM auto-detects the format from the extension)
<code>partition_method</code>	string	optional	<code>metis</code>	MPI partitioning method (<code>metis</code> or <code>cartesian</code>)

```
mesh:
  type: external
  file: "model.exo"           # GMSH(.msh) / Exodus(.exo) etc. Auto-detected
                               from extension
  partition_method: metis     # metis or cartesian
```

When using the ExodusII format (CUBIT), MFEM must be built with `MFEM_USE_NETCDF=ON`.

4.7.4 Pre-Partitioned Mesh (type: prepart_mfem)

Loads an MFEM-format mesh that has been pre-partitioned by the `partition_mesh` tool. Each MPI rank reads only its own partition file. This is useful for large meshes or when running the same mesh repeatedly.

Key	Type	Required	Default	Description
<code>prepart_mfem.directory</code>	string	required	—	Directory containing the partition files
<code>prepart_mfem.nparts</code>	int	required	—	Number of partitions (must match the number of MPI processes)

The file naming convention is `mesh.mesh.000000`, `mesh.mesh.000001`, ...

4.7.5 Configuration Examples for the mesh Section

Internal mesh:

```
mesh:
  type: internal
  origin: [0.0, 0.0, 0.0]
  size: [5000.0, 5000.0, 3000.0]
  elements: [50, 50, 30]
  max_freq: 20.0
  ppw: 5.0
  partition_method: cartesian
  partition_grid: [4, 4, 2]
```

External mesh:

```
mesh:
  type: external
  file: "./meshes/model.exo"
  max_freq: 15.0
  ppw: 5.0
```

Pre-partitioned mesh:

```
mesh:
  type: prepart_mfem
  prepart_mfem:
    directory: "./meshes/partitioned"
    nparts: 32
  max_freq: 15.0
  ppw: 5.0
```

4.8 material Section

4.8.1 Material Types and Formats

Value of type	Description
<code>isotropic_acoustic</code>	Isotropic acoustic medium (vp, rho)
<code>isotropic_elastic</code>	Isotropic elastic medium (vp, vs, rho)
<code>coupled</code>	Fluid–solid coupled medium. The fluid side is <code>isotropic_acoustic</code> , and the solid side is <code>isotropic_elastic</code> . The two are declared independently in <code>fluid:</code> and <code>solid:</code> sub-blocks. See §4.8.8 for details.

Value of format	Description
<code>constant</code>	Homogeneous material properties specified directly in YAML
<code>grid</code>	ASCII file on a structured grid
<code>by_attribute</code>	Text file keyed by mesh attribute
<code>by_attribute_mixed</code>	YAML file mixing constant/grid specifications per attribute
<code>adios2</code>	ADIOS2 BP file (for FWI)

When `type: coupled`, `format` is not specified at the top level. The nested `fluid:` / `solid:` sub-blocks each have their own `format`.

4.8.2 format: constant

Specifies material property values directly in YAML.

Key	Type	elastic	acoustic
vp	real	required	required
vs	real	required	—
rho	real	required	required

The units are m/s, m/s, and kg/m³ respectively.

```
material:
  type: isotropic_elastic
  format: constant
  vp: 6000.0
  vs: 3500.0
  rho: 2700.0
```

4.8.3 format: grid

Reads heterogeneous material properties on a structured grid from an ASCII file. Specify the file path with `file`.

File format:

```
Nx Ny [Nz]                # Number of grid points
dx dy [dz]                # Grid spacing (m)
x0 y0 [z0]                # Origin (m)
vp vs rho [Qkappa Qmu]   # Property values at each grid point
vp vs rho [Qkappa Qmu]   # (Nx*Ny[*Nz] rows)
...
```

For the elastic case there are 5 columns (vp, vs, rho, Qkappa, Qmu), and for the acoustic case there are 3 columns (vp, rho, Qkappa). The Q-value columns are only required when attenuation is enabled.

Data is stored in the order in which the x-direction index varies fastest (then y, and in 3D, then z after that). The correspondence between index and coordinates is as follows:

2D Row i corresponds to grid point (i_x, i_y) .

$$i = i_y \times N_x + i_x$$

$$x = x_0 + i_x \times dx, \quad y = y_0 + i_y \times dy$$

That is, the data scans N_x points in the x direction before advancing one step in the y direction.

3D Row i corresponds to grid point (i_x, i_y, i_z) .

$$i = i_z \times N_x \times N_y + i_y \times N_x + i_x$$

$$x = x_0 + i_x \times dx, \quad y = y_0 + i_y \times dy, \quad z = z_0 + i_z \times dz$$

That is, the data is scanned in the order $x \rightarrow y \rightarrow z$. Points outside the grid are clamped to the nearest boundary value.

The origin (x_0, y_0) or (x_0, y_0, z_0) is the grid's minimum-coordinate point, and should normally coincide with the mesh's origin. **Because the y-axis (2D) / z-axis (3D) is positive upward in semsws's coordinate system, the origin is located at the deepest part of the domain. Therefore the first line of the file begins with the property values at the deepest location.**

4.8.4 format: by__attribute

A text file that defines material properties per mesh element attribute. Used when attributes have been assigned in an external mesh (e.g., CUBIT).

#	attr	vp	vs	rho	[Qkappa Qmu]
1		6000.0	3500.0	2700.0	100.0 100.0
2		3000.0	2000.0	2200.0	50.0 50.0

4.8.5 format: by__attribute__mixed

An external YAML file that mixes constant (homogeneous), grid (grid file), or adios2 (per-attribute BP file) specifications per attribute. For each attribute with `mode: adios2`, `vp_file` / `rho_file` (and, for elastic, `vs_file`; and for attenuation, `qkappa_file` / `qmu_file`) are specified.

```
attributes:
  1:
    mode: constant
    vp: 5500.0
    vs: 3175.0
    rho: 2800.0
  2:
    mode: grid
    file: material_layer2.dat
```

4.8.6 format: adios2

Reads material data on GLL points generated by FWI iterations, in ADIOS2 BP format.

Key	Type	Required	Description
<code>vp_file</code>	string	required	Vp BP file path
<code>vs_file</code>	string	conditional	Vs BP file path (required for <code>type: isotropic_elastic</code>)
<code>rho_file</code>	string	required	ρ BP file path
<code>attenuation.files.qkappa</code>	string	optional	Q_κ BP file path (needed for viscoacoustic / viscoelastic)
<code>attenuation.files.qmu</code>	string	optional	Q_μ BP file path (needed for viscoelastic)

The `adios2` format is not currently supported for the `fluid:` / `solid:` sub-blocks of `type: coupled`.

4.8.7 Attenuation

Configures attenuation using the Generalized Zener Model (Standard Linear Solid: SLS).

Key	Type	Required	Default	Description
<code>attenuation.enabled</code>	bool	optional	false	Enable attenuation
<code>attenuation.f0</code>	real	conditional	1.0	Reference frequency (Hz). Required when <code>enabled=true</code>
<code>attenuation.n_units</code>	int	conditional	3	Number of SLS relaxation mechanisms. Required when <code>enabled=true</code>
<code>attenuation.qkappa</code>	real	conditional	—	Q value for the bulk modulus (required for <code>format: constant</code>)
<code>attenuation.qmu</code>	real	conditional	—	Q value for the shear modulus (required for <code>format: constant</code> with elastic media)
<code>attenuation.files.qkappa</code>	string	optional	—	Q_κ BP file (when specifying Q via a file with <code>format: adios2</code>)
<code>attenuation.files.qmu</code>	string	optional	—	Q_μ BP file (same, for elastic)

The fitting band for the Q values is $[0.1f_0, 10f_0]$. When using attenuation, `enabled`, `f0`, and `n_units` are required regardless of `format`. On the other hand, with `format: grid` or `by_attribute`, since Q values are taken from columns in the file, the scalar `attenuation.qkappa/attenuation.qmu` must not be specified (doing so results in an error). Specify `attenuation.files.qkappa/attenuation.files.qmu` only when you explicitly want to read Q from a separate BP file.

4.8.8 type: coupled (Fluid–Solid Coupling)

For simulations in which a fluid (acoustic) region and a solid (elastic) region share an interface, specify `material.type: coupled`. In this case, the top-level `material:` block has two sub-blocks, `fluid:` and `solid:`, each of which declares its own `type`, `format`, and material parameters (the two are independent; for example, the fluid can use `constant` while the solid uses `grid`).

Key	Type	Required/Optional	Description
<code>attribute</code>	<code>int</code>	required	The attribute value of the parent mesh element that identifies this sub-region. Must differ between <code>fluid</code> and <code>solid</code> .
<code>type</code>	<code>enum</code>	required	Specify <code>isotropic_acoustic</code> for <code>fluid</code> and <code>isotropic_elastic</code> for <code>solid</code> . Specifying <code>coupled</code> again inside is not allowed.
<code>format</code>	<code>enum</code>	required	Same values as for a regular material: <code>constant</code> , <code>grid</code> , <code>by_attribute</code> , <code>by_attribute_mixed</code> .
<code>vp</code> , <code>vs</code> , <code>rho</code>	<code>real</code>	conditional	Required for <code>format: constant</code> (<code>vs</code> is also required for the solid side).
<code>file</code>	<code>path</code>	conditional	Required for <code>format: grid / by_attribute / by_attribute_mixed</code> .
<code>attenuation</code>	<code>block</code>	optional	When an <code>attenuation:</code> sub-block is present, the same key structure as for uncoupled materials (§4.8.7) is used. Enabling it on the <code>fluid:</code> side yields viscoacoustic behavior, and on the <code>solid:</code> side yields viscoelastic behavior.

The boundary attribute for the fluid–solid interface is automatically detected at run time by `FluidSolidInterface::Setup` (among the boundary attributes present on only one of the two `ParSubMesh` instances, those not included in the outer boundary originally present in the parent mesh are extracted as the interface). There is no need to specify it explicitly in YAML.

format values usable in coupling sub-blocks. The `format` values that can be specified in the `fluid:` / `solid:` sub-blocks are limited to the four: `constant` / `grid` / `by_attribute` / `by_attribute_mixed`. `adios2` is not currently supported in coupling sub-blocks (it is accepted by the schema but aborts at run time).

```
# 2D fluid-solid coupling example. Both sides use homogeneous
# properties, with attenuation (Q_kappa = Q_mu = 40) enabled in each region
.
material:
  type: coupled

  fluid:
    attribute: 1
    type: isotropic_acoustic
    format: constant
    vp: 1500.0
    rho: 1000.0
    attenuation:
      enabled: true
      f0: 5
      n_units: 3
```

```

    qkappa: 40

solid:
  attribute: 2
  type: isotropic_elastic
  format: constant
  vp: 3000.0
  vs: 1732.0
  rho: 2500.0
  attenuation:
    enabled: true
    f0: 5
    n_units: 3
    qkappa: 40
    qmu: 40

```

Mesh-side requirements. The parent mesh must tag all fluid elements with the value of `fluid.attribute`, and all solid elements with the value of `solid.attribute` (in Cubit / GMSH this is distinguished via physical groups). In addition, the fluid elements and solid elements must be topologically connected at the interface.

4.8.9 Model Export

Model export is not performed via keys inside `material:`, but with the dedicated tool `semsws_export_model`. At run time, specify the configuration file with `-config <config.yaml>` and the output destination with `-out <output directory>`. The output is a set of ADIOS2 BP files: `vp.bp` / `vs.bp` / `rho.bp` (and, when attenuation is enabled, `qkappa.bp` / `qmu.bp`).

4.8.10 Configuration Examples for the material Section

Homogeneous elastic medium (with attenuation):

```

material:
  type: isotropic_elastic
  format: constant
  vp: 6000.0
  vs: 3500.0
  rho: 2700.0
  attenuation:
    enabled: true
    f0: 2.0
    n_units: 3
    qkappa: 100.0
    qmu: 50.0

```

Per-attribute definition (when using an external mesh):

```

material:
  type: isotropic_elastic

```

```

format: by_attribute
file: "materials.txt"
attenuation:
  enabled: true
  f0: 2.0
  n_units: 3

```

Heterogeneous acoustic medium (grid file):

```

material:
  type: isotropic_acoustic
  format: grid
  file: "velocity_model.dat"

```

4.9 boundary Section

4.9.1 Absorbing Boundary Condition (absorbing)

Configures an absorbing boundary condition using a Cerjan sponge layer. The **absorbing** block itself is optional; if omitted, there is no absorbing boundary (equivalent to **sides**: []).

Key	Type	Required	Default	Description
sides	string array	optional	[]	Sides to which it applies (disabled with the empty array []).
thickness	real	conditional	—	Sponge layer thickness (m). Required when sides is non-empty
alpha	real	conditional	—	Damping coefficient (non-negative). Required when sides is non-empty

Side names that can be specified in **sides**:

Dimension	Available sides
2D	bottom, right, top, left
3D	bottom, front, right, back, left, top

Correspondence between side names and boundary attribute IDs. In SEMSW, each side name is mapped to a fixed, 1-based MFEM boundary attribute ID. The same table is used both for **sides** and for the **dirichlet** list. *When creating a mesh with an external mesh generator such as Cubit or GMSH, this ID must be assigned to the boundary faces exactly as given.* If the IDs are mismatched, the side names on the YAML side will not correspond to the correct physical boundary.

Table 4.1: Correspondence between side names and the MFEM boundary attribute IDs used by SEMSW. These are the IDs generated by `Mesh::MakeCartesian2D` and `MakeCartesian3D`, and the same values must be followed when tagging faces while building an external mesh.

Side name	2D attribute ID	3D attribute ID
<code>bottom</code>	1	1 (z minimum)
<code>right</code>	2	3 (x maximum)
<code>top</code>	3	6 (z maximum)
<code>left</code>	4	5 (x minimum)
<code>front</code>	—	2 (y minimum)
<code>back</code>	—	4 (y maximum)

The `dirichlet` list accepts a mix of both side names and integer attribute IDs. Side names are merely aliases for the integer IDs in the table above.

4.9.2 Dirichlet Boundary Condition (`dirichlet`)

Key	Type	Required	Default	Description
<code>dirichlet</code>	array	optional	<code>[]</code>	Side names or integer boundary attribute IDs (e.g., <code>dirichlet: [top, left]</code>). The old-style <code>attributes: wrapper</code> has been removed

Side names (`top`, `left`, etc.) can be mixed with integer attribute IDs. Assigning IDs requires external mesh software (CUBIT/GMSH, etc.). With an internal mesh, only the boundaries of the mesh region can be specified, using `top`, `bottom`, etc. In SEM, unspecified boundaries become natural boundary conditions, so `dirichlet` is required for acoustic wave simulations.

4.9.3 Configuration Example for the boundary Section

```
# Offshore simulation: sea surface as Dirichlet, others as absorbing
  boundaries
boundary:
  absorbing:
    sides: [bottom, left, right]
    thickness: 1200.0
    alpha: 5.0
  dirichlet: [top]
```

4.10 sources Section

4.10.1 Source Modes and Input Formats

Key	Type	Required	Default	Description
<code>list</code>	array	conditional	—	Inline source definitions (mutually exclusive with <code>file</code>)
<code>file</code>	string	conditional	—	SEMSWS HDF5 v2.0 file path (mutually exclusive with <code>list</code>)
<code>shot_id</code>	int	optional	0	Target shot ID when <code>file</code> is specified (≥ 0)

All sources listed in `sources.list` fire simultaneously (superimposed) within the same simulation. There is no configuration key for selecting a mode in which each source fires individually.

Inline definition (`list:`). Sources are written directly in the inline `sources.list`.

HDF5 file (`file:`). When `file` is specified, source information for one shot, selected by `shot_id`, is read from a SEMSWS HDF5 v2.0 schema file. Only one of `list` and `file` should be specified; specifying both results in an error. For the detailed schema, constraints, and usage, see §4.14.

4.10.2 Common Source Parameters (`list`)

Specify the following for each element of the `sources.list` array:

Key	Type	Required	Default	Description
<code>id</code>	int	optional	<code>i+1</code>	Source ID (if unspecified, automatically numbered based on position in the array)
<code>name</code>	string	optional	—	Source name (unused in the current version)
<code>type</code>	string	required	—	<code>force</code> , <code>pressure</code> , <code>moment_tensor</code>
<code>location</code>	real array	required	—	Location <code>[x,y]</code> or <code>[x,y,z]</code> (m)

4.10.3 Source Type: `force`

A single force for elastic wave simulations.

Key	Type	Required	Description
<code>direction</code>	real array	required	Force direction vector <code>[x,y]</code> or <code>[x,y,z]</code>

```
- id: 1
  type: force
  location: [500.0, 300.0, 0.0]
  direction: [0.0, 0.0, 1.0]      # Vertical direction
```

```

wavelet:
  type: ricker
  frequency: 10.0
  amplitude: 1.0e10

```

4.10.4 Source Type: pressure

A pressure point source for acoustic wave simulations. No additional parameters.

4.10.5 Source Type: moment_tensor

A moment tensor source. Specifies the earthquake mechanism solution directly.

In 3D (6 components): Mxx, Myy, Mzz, Mxy, Mxz, Myz.

In 2D (3 components): Mxx, Myy, Mxy.

```

- id: 1
  type: moment_tensor
  location: [5000.0, 5000.0, 3000.0]
  moment_tensor:
    Mxx: 1.0e18
    Myy: 0.0
    Mzz: 0.0
    Mxy: 0.0
    Mxz: 0.0
    Myz: 0.0
  wavelet:
    type: ricker
    frequency: 5.0
    amplitude: 1.0

```

4.10.6 Wavelet

Key	Type	Required	Default	Description
type	string	optional	ricker	ricker, gaussian, external
frequency	real	conditional	—	Dominant frequency (Hz). Required for ricker/-gaussian
amplitude	real	conditional	—	Amplitude scaling. Required for ricker/gaussian (ignored for external, fixed at 1.0)
delay	real	optional	0.0	Delay of the peak arrival time (s)
file	string	conditional	—	External STF file path (required when external)

For the Ricker wavelet, $\text{delay} \geq 1.2/\text{frequency}$ is recommended.

The external STF file is 2-column ASCII text (time, amplitude) (the time column is ignored, so the file must be arranged so that the amplitude on the first line corresponds to the simulation start time):

```
# time (s)    amplitude
0.0000       0.000000e+00
0.0001       1.234568e-05
...
```

4.10.7 Observed Data — Inversion Mode

When `simulation.mode` is `inversion` or `misfit_only`, observed data is read from the SEMSWS HDF5 v2.0 file specified by `sources.file` (for the one shot selected by `sources.shot_id`). In this mode, `sources.file` is required. There is no dedicated `observed:` key.

When using SU/SEG-Y, etc. as observed data, convert it beforehand to SEMSWS HDF5 v2.0 format.

4.10.8 Configuration Examples for the sources Section

Acoustic wave (multiple sources, simultaneous firing):

```
sources:
  list:
    - id: 1
      type: pressure
      location: [300.0, 400.0]
      wavelet:
        type: ricker
        frequency: 30.0
        amplitude: 1.0e10
        delay: 0.05
    - id: 2
      type: pressure
      location: [700.0, 400.0]
      wavelet:
        type: ricker
        frequency: 30.0
        amplitude: 1.0e10
        delay: 0.05
```

Elastic wave (moment tensor, external STF):

```
sources:
  list:
    - id: 1
      type: moment_tensor
      location: [5000.0, 5000.0, 3000.0]
      moment_tensor:
        Mxx: 1.0e18
        Myy: -1.0e18
        Mzz: 0.0
        Mxy: 0.0
```

```

    Mxz: 0.0
    Myz: 0.0
  wavelet:
    type: external
    frequency: 5.0
    file: "stf_earthquake.txt"

```

When using an external HDF5 file, specify the HDF5 file with `file` (mutually exclusive with `list`):

```

sources:
  file: "./sources.h5"
  shot_id: 0

```

4.11 receivers Section

4.11.1 Ways to Specify Receivers

Receivers can be specified in the following two ways, with the recording type (`type`) and output settings (`output`) common to both.

Inline (`list` / `line`). Written directly in YAML. Use `list` to enumerate receivers individually, or `line` to automatically generate an evenly spaced line of receivers.

HDF5 file (`sources.file`). When `sources.file` is specified, the receiver positions and types within that file are used (mainly for FWI). In this case the `receivers` section can be omitted. For the detailed schema and usage, see §4.14.

4.11.2 Recording Type (`type`) and Output Settings (`output`)

Key	Type	Required	Default	Description
<code>type</code>	string/array	required	—	Default recording type (can be overridden per receiver; see below)
<code>output.formats</code>	array	required	—	List of output formats (plain strings <code>ascii/hdf5/su</code>)
<code>output.filename</code>	string	optional	<code>shot</code>	File name for hdf5/su (ignored for ascii)
<code>output.buffer_steps</code>	int	optional	—	Number of steps to buffer output before flushing (≥ 0)

Values that can be specified in `type`:

Value	Description	Applicable model
DISP	Displacement	elastic
VEL	Velocity	elastic
ACC	Acceleration	elastic
PS	Pressure	acoustic
GRAD	Strain (gradient, forward simulation only)	elastic

Output formats. `output.formats` is a list of strings, in which each element is written directly as one of `ascii` (text, one file per receiver), `hdf5` (single HDF5 file), or `su` (Seismic Unix format). Multiple formats can be requested simultaneously:

```
output:
  formats:
    - ascii
    - su
  filename: "seismograms"      # File name for hdf5 / su (optional, default
                                shot)
```

To visualize the output su file with Seismic Unix, Seismic Unix must be built with XDRFLAG turned OFF.

4.11.3 Receiver List (list)

`list` enumerates each receiver. The keys of each element are as follows.

Key	Type	Required	Default	Description
<code>name</code>	string	required	—	Receiver name
<code>location</code>	real array	required	—	Location [x,y] or [x,y,z] (m)
<code>weight</code>	real	optional	1.0	Scalar weight (used when assembling FWI observed data)

4.11.4 Receiver Line (line)

Automatically generates an evenly spaced line of receivers.

Key	Type	Required	Default	Description
<code>line.start</code>	real array	required	—	Start position (m)
<code>line.end</code>	real array	required	—	End position (m)
<code>line.count</code>	int	required	—	Number of receivers

4.11.5 Configuration Examples for the receivers Section

List form (individual specification):

```

receivers:
  type: [DISP, VEL]
  output:
    formats:
      - ascii
      - su
    filename: "seismograms"
  list:
    - name: "R01"
      location: [1000.0, 2500.0, 0.0]
    - name: "R02"
      location: [2000.0, 2500.0, 0.0]
    - name: "R03"
      location: [3000.0, 2500.0, 0.0]

```

Line form (automatic even spacing):

```

receivers:
  type: [PS]
  output:
    formats:
      - ascii
  line:
    start: [100.0, 300.0]
    end: [900.0, 300.0]
    count: 17

```

Example of receiver placement for fluid–solid coupling. Specifying the union of all types ([PS, DISP, VEL, ACC]) means that each receiver automatically records only the types that are physically valid on the side where it is located (R001 is on the fluid side, so only PS; R002/R003 are on the solid side, so only DISP/VEL/ACC. Types that don't match the side are automatically excluded, and this is logged):

```

receivers:
  output:
    formats:
      - ascii
  type: [PS, DISP, VEL, ACC]
  list:
    - name: "R001"
      location: [800.0, 200.0]
    - name: "R002"
      location: [800.0, 440.0]
    - name: "R003"
      location: [800.0, 680.0]

```

4.12 device section

Key	Type	Required	Default	Description
<code>type</code>	string	Required	—	<code>cpu</code> , <code>omp</code> , <code>cuda</code> , <code>hip</code> , <code>raja-cpu</code> , <code>raja-omp</code> , <code>raja-cuda</code> , <code>raja-hip</code>

`type` must match the build configuration. Specifying `cpu` in a GPU build will work but is extremely slow, so a CPU build should be used for CPU execution.

The number of steps buffered for the waveform recording buffer on the GPU is specified with `receivers.output.buffer_steps` (not in the `device` section).

4.12.1 Example device section configuration

```
device:
  type: cuda
```

4.13 inversion section

An optional top-level section that can be placed when `simulation.mode` is `inversion` or `misfit_only`. It switches the computation path used to derive sensitivity kernels.

All keys are under `inversion:`. Defaults are noted in the description column.

Key	Type	Required	Description
<code>sensitivity_backend</code>	string	Optional	Sensitivity kernel backend (default <code>hand</code>). <code>hand</code> (hand-written adjoint path, production) or <code>ad</code> (verification path using forward-mode automatic differentiation)
<code>misfit_type</code>	string	Optional	Misfit function (default <code>l2_waveform</code>). <code>l2_waveform</code> or <code>normalized_correlation</code>
<code>kernel_dir</code>	string	Optional	Sensitivity kernel output directory (default <code><simulation.directory>/kernels</code>)
<code>checkpoints.n</code>	int	Optional	Number of checkpoints (default 10, ≥ 1)
<code>checkpoints.storage</code>	string	Optional	Checkpoint storage location (default <code>memory</code>). <code>memory</code> or <code>disk</code> (<code>dir</code> required when <code>disk</code>)
<code>checkpoints.device</code>	string	Optional	Checkpoint device (default <code>auto</code>). <code>auto</code> / <code>host</code> / <code>device</code>
<code>checkpoints.dir</code>	string	Optional	Storage directory when <code>storage: disk</code> (default <code><simulation.directory>/checkpoints</code>)

`ad` is an alternative path for verifying that the hand-written implementation (`hand`) matches numerically; it is computationally expensive. Normally the default `hand` should be used.

```
inversion:
  sensitivity_backend: hand    # or "ad" for AD verification
```

4.14 HDF5 I/O Schema and Usage

In addition to YAML, SEMSWs supports an HDF5 v2.0 schema path, separate from YAML, for source, receiver, and observed-data input, and for receiver waveform output. For multi-shot FWI workflows (where `semsws-driver` loops on the outside) and for inversion modes that read observed data, HDF5 is effectively a required input format.

4.14.1 Overview of the HDF5 v2.0 schema (common)

Every input/output file has `format_version="2.0"` as a root attribute, and groups the sources and receivers for one shot under `/shots/<NNNN>/` (`<NNNN>` is the 4-digit zero-padded shot ID).

```
/
    attrs: format_version="2.0",
           dt, t0, n_samples, space_dim,
           coord_system="cartesian", units="
           SI"
/shots/<NNNN>/
  /sources/<key>/
    attrs: shot_id (i32)
           id (i32), type (str),
           position (f64[space_dim]),
           direction (f64[space_dim]; for
           force),
           label (str; optional)
  /moment_tensor
    (6,) or (3,)          f64    # for
    type=moment_tensor
    attrs: component_order (var-length str
    array; required)
  /stf
    (n_samples,)          f64    # time-
    series STF (F5: f64 required, shape=(n_samples,))
/receivers/<key>/
  attrs: position (f64[space_dim])
         types (var-length str array;
         optional)
         label (str; optional)
  /PS
    (nt,)                  f32    #
    waveform for type=PS
  /VEL
    (space_dim, nt)        f32    #
    waveform for type=VEL
  /DISP
    (space_dim, nt)        f32    #
    waveform for type=DISP
  /ACC
    (space_dim, nt)        f32    #
    waveform for type=ACC
  /weight_<ch>
    same shape as <ch>     f32    #
    optional, weight on the observed side (<ch>=PS/VEL/DISP/ACC)
```

Relationship between shot and source. `/shots/<NNNN>/` represents a single firing event (i.e., a single simulation), and all sources belonging to `/sources/` under it **fire simultaneously**. The receivers record only a single set of combined waveforms summed over all sources. There is no key (such as a `sources.mode`) for switching to sequential firing. To record sources separately, split them into different shots by changing `shot_id` (handled by the shot loop in `semsws-driver`).

Group name <key> convention.

- **Sources:** `S<NNNN>` (4-digit zero-padded). Must match the `@id` attribute (`@id=42` implies `S0042`). The reader validates this strictly.
- **Receivers:** any string is allowed (no naming convention). Each receiver's **name** is taken from the `@label` attribute if present, otherwise from the group name, and is used for (1) the ASCII output file name (`<name>_<shot_id>.p`, etc.), (2) the `/receivers/<name>/` group name on HDF5 output, and (3) log/diagnostic messages.

Description of attributes and datasets. Below, the type `f64[d]` denotes a vector of length `d=space_dim` (spatial dimension).

Table 4.2: Root attributes

Attribute	Type	Required	Description
<code>format_version</code>	<code>str</code>	Required	Schema version ("2.0")
<code>dt</code>	<code>f64</code>	Required	Time step (s)
<code>t0</code>	<code>f64</code>	Required	Start time (s)
<code>n_samples</code>	<code>i64</code>	Required	Number of time-series samples
<code>space_dim</code>	<code>i32</code>	Required	Spatial dimension (2 or 3)

Table 4.3: Shot and source attributes and datasets

Attribute / Dataset	Type	Required	Description
@shot_id	i32	Required	Attribute of the <code>/shots/<NNNN>/</code> group. Identifier of the shot (firing event). Reflected in the output file name suffix and the <code>/shots/<NNNN>/</code> key
@id	i32	Required	Source ID (positive integer, unique within the shot). Must match the group name <code>S<NNNN></code>
@type	str	Required	Source type: <code>force</code> / <code>pressure</code> / <code>moment_tensor</code>
@position	f64[d]	Required	Coordinates of the source location
@direction	f64[d]	Conditional	Direction of the force (required when <code>type=force</code>)
@label	str	Optional	Display name (arbitrary metadata)
/moment_tensor	f64 (6,)/(3,)	Conditional	Moment tensor components (required when <code>type=moment_tensor</code>). Accompanied by <code>@component_order</code> (required), which indicates the component name for each value
/stf	f64 (n_samples,)	Required	Time series of the source time function

Table 4.4: Receiver attributes and datasets

Attribute / Dataset	Type	Required	Description
@position	f64[d]	Required	Coordinates of the receiver location
@types	str array	Optional	Channels recorded by this receiver (<code>PS/VEL/DISP/ACC</code>). If omitted, inherits the parent <code>receivers.type</code> ; an error occurs if neither is present
@label	str	Optional	Display name (see "group name convention" above)
/PS	f32 (nt,)	—	Pressure scalar waveform (when <code>types</code> includes <code>PS</code>)
/VEL, /DISP, /ACC	f32 (d, nt)	—	Vector waveforms of velocity, displacement, and acceleration
/weight_<ch>	f32	Optional	Per-channel weight on the observed side (<code><ch>=PS/VEL/...</code>)

Specifying the moment tensor (`component_order`). For a source with `type=moment_tensor`, the independent-component values are stored in `/moment_tensor` as a 1-D array, and a string array `@component_order` of the same length declares which component each value represents. Valid component names are, in 3D, `Mxx`, `Myy`, `Mzz`, `Mxy`, `Mxz`, `Myz` (6 values), and in 2D, `Mxx`, `Myy`, `Mxy` (3 values). **The order is free**, as long as `@component_order` corresponds position-by-position to the data array. All components must be included **exactly once**; duplicates, invalid names, or a count mismatch result in an error.

4.14.2 Input path: HDF5 → SEMSWS

Passing sources from HDF5 (`sources.file`)

```
sources:
  file: inputs/shots.h5      # <- specifying file selects the HDF5 input
                             path (there is no format key)
  shot_id: 0                 # reads /shots/0000/sources/
```

`sources.shot_id` specifies the shot number to read. Since the source's `type` / `position` / `direction` / `moment_tensor` and the STF time series are all obtained from HDF5, the YAML-side `sources.list` is not written (`sources.file` and `sources.list` are mutually exclusive; specifying both is an error).

Important: on the `sources.file` path, the STF **must** be stored as time-series data in the `/shots/<NNNN>/sources/<key>/stf` dataset. Specifying an analytic wavelet via YAML, such as `wavelet.type: ricker`, cannot be combined with `sources.file`. Even to use a Ricker wavelet, etc., the corresponding discrete time series must be precomputed and written into the HDF5 file beforehand.

Passing receiver locations from HDF5 (automatically obtained from `sources.file`)

```
# Receiver locations are automatically obtained from the HDF5 file
#   specified by sources.file
# (there is no file/format/shot_id key on the receivers side).
sources:
  file: inputs/shots.h5
  shot_id: 0                 # /shots/0000/receivers/ is also read from
                             the same file and same shot_id
receivers:
  output:
    formats: [ascii]
    filename: "seis"
```

When `sources.file` is specified and neither `list:` nor `line:` is present on the `receivers` side, receiver locations are obtained from the same HDF5 file (`/shots/<NNNN>/receivers/`). Only each receiver's `@position` / `@types` / `@label` is read on this path; waveform datasets such as `/PS` / `/VEL` are not required (since geometric information alone is sufficient for a forward run). These waveform datasets are only needed when read as observed data during inversion.

Observed data (inversion only)

When `simulation.mode: inversion` or `misfit_only`, the observed waveform for each source is specified as follows:

```
# In inversion / misfit_only mode, observed data is also read from the HDF5
# file
# specified by sources.file (the PS/VEL/DISP/ACC datasets under
# /shots/<shot_id>/receivers/<key>/). There is no dedicated observed: block
.
sources:
  file: obs/shot_0000.h5          # v2.0 file containing sources,
    receivers, and observed waveforms (required)
  shot_id: 0
```

The observed file follows the schema in §4.14.1 and references the datasets (PS, VEL, DISP, ACC) under `/shots/<NNNN>/receivers/<key>/`. If the receiver's `@position` attribute does not match the receiver location on the simulation side, the misfit (residual) will not be computed correctly.

Examples of input HDF5 structures

Three typical examples of input HDF5 contents are shown by use case. In all cases, root attributes (`format_version=2.0`, `dt`, `t0`, `n_samples`, `space_dim`, ...) are omitted for brevity.

Example 1: Single source, single shot (typical ocean-bottom seismometer simulation).

```
inputs/shot.h5
/shots/0000/          attrs: shot_id=0
  /sources/
    /S0001/          attrs: id=1, type="pressure",
                      position=[5000, 4000]
                      (n_samples,) <- precomputed time
                      series
  /receivers/
    /R001/  attrs: position=[1000, 0]
    /R002/  attrs: position=[2000, 0]
    /R003/  attrs: position=[3000, 0]
```

Corresponding YAML:

```
sources:
  file: inputs/shot.h5      # v2.0 file containing sources and receivers
  shot_id: 0
receivers:
  type: [PS]                # default type for all receivers (overridable
    via @types)
```

Example 2: Multiple shots, multiple sources within a shot.

```

inputs/multishot.h5
/shots/0000/                                attrs: shot_id=0
/sources/
/S0001/   attrs: id=1, type="force", direction=[1,0,0]
/stf
/S0002/   attrs: id=2, type="moment_tensor",
           moment_tensor=[Mxx,Myy,Mzz,Mxy,Mxz,Myz]
/stf
/receivers/
/R001/   attrs: position=[...], types=["DISP","VEL"]
/R002/   attrs: position=[...], types=["VEL"]
/shots/0001/                                attrs: shot_id=1
/sources/
/S0010/   attrs: id=10, type="pressure"
/stf
/receivers/
/R001/   attrs: position=[...], types=["DISP","VEL"]
/R002/   attrs: position=[...], types=["VEL"]
/shots/0002/                                attrs: shot_id=2
...

```

The driver launches SEMSWS sequentially while switching `shot_id=0,1,2,...`. Within each shot, multiple sources always fire simultaneously. The per-receiver `@types` attribute allows the recorded channels to vary by station.

4.14.3 Output path: SEMSWS → HDF5**Self-roundtrip specification and driver merge**

Self-roundtrip. Each output `seis<NNNN>.h5` contains both `/sources/` and `/receivers/`, so it can be reused directly as input to SEMSWS. For example, using a forward result as observed data for a subsequent inversion, or as a reference file for a forward reproduction test, etc.

When using `semsws-driver`, the driver automatically builds a per-shot directory layout:

```

output_root/
shot_0000/                                <- processing results for input shot 0
  config.yaml                             (YAML rewritten for shot 0, shot_id=0)
  seismograms0000.h5                     (internal /shots/0000/, suffix also 0000)
  stdout.log
shot_0001/                                <- input shot 1
  config.yaml                             (shot_id=1)
  seismograms0001.h5                     (internal /shots/0001/, suffix 0001)
  stdout.log
shot_0002/
  config.yaml                             (shot_id=2)
  seismograms0002.h5                     (internal /shots/0002/, suffix 0002)
  stdout.log

```

Driver merge tool. When `semsws-driver` runs multiple shots sequentially, a post-process is provided to combine the resulting per-shot file set into a single HDF5 file (corresponding to `driver/tools/merge_shot_outputs.py`). Example output:

```
shots_merged.h5
/shots/
  /0000/  (sources + receivers for shot 0)
  /0001/  (... for shot 1)
  /0002/  (...)
```

Since the `/shots/<NNNN>/` in each per-shot file already lands at the correct index, the merge tool does not need to renumber `shot_id`; a simple concatenation suffices.

Example output structure (YAML configuration → generated HDF5)

Example 1: Single source (typical forward simulation). YAML:

```
sources:
  shot_id: 0                      # defaults to 0 if omitted
  list:
    - id: 1
      type: pressure
      location: [5000, 4000]
      wavelet: { type: ricker, frequency: 5.0, amplitude: 1.0, delay: 0.4 }
receivers:
  type: [PS]
  output:
    formats: [hdf5]
    filename: "seis"
  list:
    - { name: "R001", location: [1000, 0] }
    - { name: "R002", location: [2000, 0] }
```

Generated file (one):

```
out/seis0000.h5                                <- suffix = input shot_id=0
/shots/0000/                                   attrs: shot_id=0
  /sources/
    /S0001/  attrs: id=1, type="pressure", position=[5000,4000]
      /stf                                (n_samples,)
  /receivers/
    /R001/   attrs: position=[1000,0]
      /PS                                (nt,)
    /R002/   attrs: position=[2000,0]
      /PS                                (nt,)
```

Example 2: Multiple sources (simultaneous firing, combined waveform in a single file). YAML:

```
sources:
```

```

shot_id: 7                                # <- assuming input shot_id=7
list:
  - id: 1; type: force; location: [...]; direction: [1,0]; wavelet: {...}
  - id: 2; type: force; location: [...]; direction: [0,1]; wavelet: {...}
  - id: 3; type: pressure; location: [...]; wavelet: {...}
receivers:
  type: [DISP, VEL]
  output:
    formats: [hdf5]
    filename: "seis"
  list:
    - { name: "R001", location: [...] }

```

Generated file (combined result of 3 sources in a single HDF5 file):

```

out/seis0007.h5                                <- suffix = input shot_id=7
/shots/0007/                                  attrs: shot_id=7
  /sources/                                    <- metadata for all active sources
    /S0001/ (force, /stf)
    /S0002/ (force, /stf)
    /S0003/ (pressure, /stf)
  /receivers/
    /R001/                                     <- combined waveform of 3 simultaneous
      sources
      /DISP                                     (space_dim, nt)
      /VEL                                     (space_dim, nt)

```

Important: even with 3 sources, there is only 1 HDF5 file. Each receiver's waveform dataset contains the combined result of the 3 sources. Both the file name suffix and the internal `/shots/<NNNN>/` reflect the input `shot_id`.

Chapter 5

Tools and Scripts

In addition to the simulation executable itself (`semsws`), SEMSWS is distributed with preprocessing tools for mesh partitioning, refinement, and quality evaluation, as well as Python scripts for visualizing results. This chapter explains how to use them.

5.1 `partition_mesh` — Pre-partitioning a Mesh

`partition_mesh` is a tool that pre-partitions a mesh file for MPI parallel execution (serial run only). Using a pre-partitioned mesh avoids the overhead of loading and partitioning the mesh at simulation startup. It is useful when running a simulation multiple times with the same mesh and the same number of partitions.

5.1.1 Usage

```
# Partition an external mesh into 64 parts with METIS
./build/src/partition_mesh -n 64 domain.exo partitions/

# Cartesian partitioning (4*4*4 = 64)
./build/src/partition_mesh -n 64 -p cartesian -g 4,4,4 domain.exo
    partitions/

# Generate and partition an internal mesh
./build/src/partition_mesh -n 64 --internal \
    --size 10000,10000,5000 --elements 100,100,50 partitions/
```

5.1.2 Options

Option	Description
<code>-n <nparts></code>	Number of partitions (required)
<code>-p <method></code>	Partitioning method: <code>metis</code> (default) or <code>cartesian</code>
<code>-g <nx,ny,nz></code>	Grid dimensions for Cartesian partitioning
<code>--internal</code>	Internal mesh generation mode
<code>--dim <2 3></code>	Dimension (for internal mesh, default: 3)
<code>--origin <x,y,z></code>	Origin (for internal mesh)
<code>--size <lx,ly,lz></code>	Domain size (for internal mesh, required)
<code>--elements <nx,ny,nz></code>	Number of elements (for internal mesh, required)

Output files are generated in the specified directory as `mesh.mesh.000000`, `mesh.mesh.000001`, ...

5.1.3 Usage in config.yaml

To use a pre-partitioned mesh in a simulation, configure the `mesh` section of `config.yaml` as follows:

```
mesh:
  type: prepart_mfem
  prepart_mfem:
    directory: "partitions/"
    nparts: 64
```

`nparts` must match the number of processes used at MPI runtime.

In addition to the partition files such as `mesh.mesh.000000`, the output directory also contains `partition_info.yaml` (metadata such as the number of partitions and element counts; not required for loading).

5.2 combine_mesh — Combining Partitioned Meshes

`combine_mesh` is a tool that combines a mesh partitioned by `partition_mesh` into a single file. It is used for visualization with GLVis and for checking partitioned meshes.

```
# Combine a mesh partitioned into 64 parts (run with the same np as the
# partition count)
mpirun -np 64 ./build/src/combine_mesh partitions/ combined.mesh
```

The number of MPI processes at runtime must match the number of partitions.

The combined mesh can be checked with GLVis:

```
glvis -m combined.mesh
```

5.3 evaluate_mesh — Mesh Quality Evaluation

`evaluate_mesh` is a tool that evaluates mesh quality and checks its consistency with the simulation parameters. It computes statistics for element size, the CFL condition, PPW (Points Per Wavelength), aspect ratio, and the Jacobian determinant.

5.3.1 Usage

```
# Basic evaluation (console output only)
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml

# Also output histogram CSV files
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml \
  --histogram mesh_hist --nbins 50
```

5.3.2 Options

Option	Description
<code>--config <file></code>	Configuration file (required)
<code>--cfl <factor></code>	Override the CFL factor
<code>--histogram <prefix></code>	Output prefix for histogram CSV files
<code>--nbins <N></code>	Number of histogram bins (default: 50)

5.3.3 Histograms Output

When `--histogram` is specified, the following CSV files are generated:

File	Content
<code><prefix>_element_size.csv</code>	Element size distribution
<code><prefix>_dt_cfl.csv</code>	Distribution of $\Delta t_{\max}$ from the CFL condition
<code><prefix>_ppw.csv</code>	PPW distribution
<code><prefix>_aspect_ratio.csv</code>	Aspect ratio distribution
<code><prefix>_jacobian_det.csv</code>	Jacobian determinant distribution

5.4 refine_mesh — Uniform Mesh Refinement and Partitioning

`refine_mesh` is a tool that uniformly refines a mesh based on material properties. It automatically splits elements in low-velocity regions (regions with short wavelengths), generating a mesh that satisfies the PPW condition.

The refinement criterion is determined by the following equation. First, the required element size h_{required} is computed:

$$h_{\text{required}} = \frac{v_{\min} \times N_{\text{GLL}}}{f_{\max} \times \text{PPW}}$$

where $v_{\min}$ is the minimum wave velocity over all elements, $N_{\text{GLL}} = \text{order} + 1$, and $f_{\max}$ and PPW are specified by `mesh.max_freq` and `mesh.ppw` in the config.

Next, the number of refinement passes n is determined from the current element size h_{current} :

$$n = \left\lceil \log_2 \frac{h_{\text{current}}}{h_{\text{required}}} \right\rceil$$

If $h_{\text{current}} \leq h_{\text{required}}$, no refinement is needed ($n = 0$). The maximum n over all elements becomes the refinement level for the entire mesh.

Note that the entire mesh is refined uniformly. Since each refinement pass splits every element in two, the number of elements increases by a factor of 2^{2n} in 2D and 2^{3n} in 3D. Consequently, some compromise on metrics such as PPW may be necessary.

There are two main use cases. One is creating a mesh that satisfies the PPW condition; the other is handling large-scale computations (large meshes) on HPC systems. **In SEMSWS, unless `mesh.type: prepart_mfem` is specified, even in parallel computation Rank 0 reads the entire mesh, then partitions it and distributes it to each rank via `MPI_Send`. This causes memory issues for large meshes. In addition, meshes pre-partitioned by tools such as CUBIT cannot be loaded, and even pre-partitioning with `partition_mesh` initially requires Rank 0 to read the entire mesh.** In such cases, one approach is to create a coarse mesh capturing the approximate shape — such as topography or bathymetry — with CUBIT or GMSH, and then refine it with `refine_mesh`. As described below, `refine_mesh` reads the input mesh on Rank 0, partitions the mesh, refines it on each rank, and then outputs the partitioned mesh. Once the output mesh is set with `mesh.type: prepart_mfem` in the config, each rank reads only the mesh for its own partition.

5.4.1 Mesh Loading and Output Behavior

`refine_mesh` internally calls `LoadParMesh()` to load the mesh. The loading path differs depending on `mesh.type` in the config:

For an unpartitioned mesh (`type: internal / external`) Only Rank 0 reads the serial mesh (or generates it, in the case of internal), partitions it into N parts with `MeshPartitioner`, and then distributes them to each rank via `MPI_Send`. N is determined by the number of processes given to `mpirun -np N`.

For a pre-partitioned mesh (`type: prepart_mfem`) Each rank independently reads its own partition file (`mesh.mesh.NNNNNN`). The number of MPI processes at runtime must match `nparts`.

Refinement and output In either case, the refinement itself runs in parallel via `ParMesh::UniformRefinement` with no difference in behavior; only the loading path differs. There are two ways to output the refined mesh:

- `--output-dir`: Each rank writes out its own partition file with `ParPrint()` (N files).
Used in a simulation as `type: prepart_mfem`

- `--save-as-one`: Rank 0 combines the mesh into a single file with `SaveAsOne()`. Used for visualization with GLVis or in a simulation as `type: external`

5.4.2 Usage

```
# Run with 8 processes (MPI required)
mpirun -np 8 ./build/src/refine_mesh --config config.yaml --output-dir MESH
/

# Also save a combined mesh for visualization
mpirun -np 8 ./build/src/refine_mesh --config config.yaml \
  --output-dir MESH/ --save-as-one refined.mesh
```

5.4.3 Options

Option	Description
<code>--config <file></code>	Configuration file (required)
<code>--output-dir <dir></code>	Output destination for the partitioned mesh (required)
<code>-n <max_ref></code>	Override the maximum refinement level
<code>--save-as-one <file></code>	Also save a combined mesh (for GLVis visualization)

The output is in the same format as `partition_mesh` (`mesh.mesh.NNNNNN` and `partition_info.yaml`), and can be used directly as `type: prepart_mfem` in `config.yaml`.

5.5 Example Workflow: Mesh Evaluation and Refinement

This section demonstrates the full workflow from mesh evaluation through refinement and re-evaluation, using the two-layer viscoelastic model in `examples/visco_elastic_3d_layered/` as an example.

5.5.1 Problem Setup

A two-layer viscoelastic medium is set up in a 3D domain of $10 \text{ km} \times 10 \text{ km} \times 10 \text{ km}$. The upper layer ($z > -4000 \text{ m}$, attribute 2) has a low velocity ($V_p = 3000 \text{ m/s}$, $Q_\kappa = Q_\mu = 50$), and the lower layer ($z \leq -4000 \text{ m}$, attribute 1) has a high velocity ($V_p = 6000 \text{ m/s}$, $Q_\kappa = Q_\mu = 100$) (these attribute numbers correspond to `Attr 1/2` in the material property file and evaluation output).

The mesh is generated with CUBIT (the free Coreform edition works). Running the included journal file `mesh_layered_3d.jou` generates a mesh in ExodusII format:

```
# Run the following in the CUBIT GUI CONSOLE
playback 'mesh_layered_3d.jou'
```

Contents of the material property file `materials_visco.txt`:

#	attribute	<i>vp</i>	<i>vs</i>	<i>rho</i>	<i>qkappa</i>	<i>qmu</i>
1		6000.0	3500.0	2700.0	100.0	100.0
2		3000.0	2000.0	2200.0	50.0	50.0

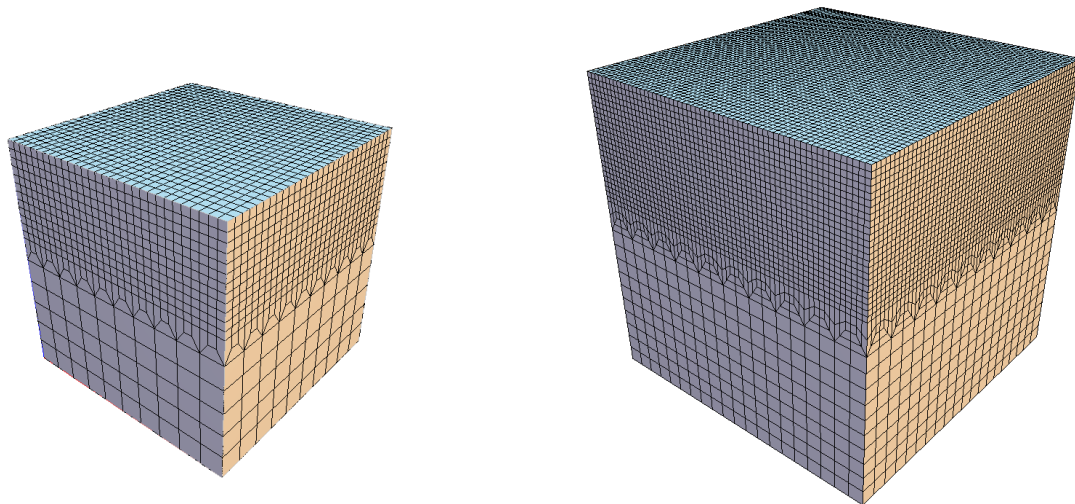


Figure 5.1: Mesh before refinement (left) and after refinement (right). Left: initial mesh generated with CUBIT (12,600 elements). Upper layer has 500 m elements, lower layer has 1,000 m elements. Right: after one uniform refinement pass with `refine_mesh` (100,800 elements). All elements have been split $2^3 = 8$ fold.

5.5.2 Step 1: Evaluating the Initial Mesh

First, evaluate the quality of the initial mesh:

```
./build/src/evaluate_mesh --config config_visco.yaml --histogram mesh
```

Sample output:

```
Mesh Evaluation Report
=====
Mesh: 12600 elements (global), dimension 3
Order: 4, CFL factor: 0.3, f_max: 5.6 Hz, PPW target: 5

Per-element statistics:
      min      max      mean      std
h [m]      3.333e+02  1.000e+03  3.858e+02  1.561e+02
dt_CFL [s]  2.141e-03  8.562e-03  5.394e-03  1.227e-03
PPW         3.163e+00  9.490e+00  5.436e+00  7.247e-01
Aspect ratio 1.000e+00  8.492e+00  1.331e+00  1.191e+00
det(J)      2.778e+07  1.000e+09  7.540e+07  1.883e+08

CFL summary:
  dt_max (global): 2.1406e-03 s

PPW summary:
```

```
Worst PPW: 3.16 at centroid (9500.00, 4500.00, -6500.00)
PPW < 5.0: 1300 / 12600 (10.32%)
```

Per-attribute breakdown:

```
Attr 1: 1800 elems, Vp=6050, Vs=3543, PPW_min=3.2, dt_min=2.14e-03
Attr 2: 10800 elems, Vp=3066, Vs=2049, PPW_min=5.5, dt_min=5.63e-03
```

The minimum PPW value of 3.16 is below the target of 5, showing that 1300 elements (10.3%) in Attr 1 (the lower layer, the high-velocity region) have insufficient PPW.

5.5.3 Step 2: Uniform Refinement

Refine the mesh with `refine_mesh`. Here it is run with 4 processes, but the included `run_visco.sh` runs it with 64 processes:

```
mpirun -np 4 ./build/src/refine_mesh \
  --config config_visco.yaml \
  --output-dir mesh/
```

Sample output:

```
Loading mesh...
Elements (global): 12600

Loading material...
V_min (global): 2049.22
V_max (global): 6050.01

Refinement levels needed: 1
Refinement 1: 100800 elements (global)

Refined mesh:
Elements (global): 100800
h range:          [84.12, 500.03]
PPW (worst):      6.33 (target: 5)
CFL dt_max:       0.00107 s (CFL=0.3)

Saving partitioned mesh to: mesh/
4 partition files (mesh.mesh.NNNNNN)
```

One refinement pass increased the 12,600 elements 8-fold to 100,800 elements, and the minimum PPW improved from 3.16 to 6.33 (Figure 5.1).

5.5.4 Step 3: Re-evaluation After Refinement

Re-evaluate the refined mesh to confirm that the PPW condition is satisfied. `config_visco_sim.yaml` references the refined mesh with `mesh.type: prepart_mfem`:

```
mpirun -np 4 ./build/src/evaluate_mesh --config config_visco_sim.yaml
```

Sample output:

```

Mesh Evaluation Report
=====
Mesh: 100800 elements (global), dimension 3
Order: 4, CFL factor: 0.3, f_max: 5.6 Hz, PPW target: 5

Per-element statistics:

              min          max          mean          std
h [m]          1.667e+02    5.000e+02    1.938e+02    8.145e+01
dt_CFL [s]      1.070e-03    4.281e-03    2.718e-03    5.622e-04
PPW             6.326e+00    1.898e+01    1.087e+01    1.563e+00
Aspect ratio    1.000e+00    8.495e+00    1.230e+00    8.541e-01

PPW summary:
Worst PPW: 6.33 at centroid (250.00, 9250.00, -9250.00)
PPW < 5.0: 0 / 100800 (0.00%)

Per-attribute breakdown:
Attr 1: 14400 elems, Vp=6050, Vs=3543, PPW_min=6.3, dt_min=1.07e-03
Attr 2: 86400 elems, Vp=3066, Vs=2049, PPW_min=11.0, dt_min=2.82e-03

```

The proportion of elements with insufficient PPW is now 0%, and all elements satisfy the condition $PPW \geq 5$.

5.5.5 Step 4: Running the Simulation

Run the simulation with the refined mesh. For `type: prepart_mfem`, the number of MPI processes **must match the number of partitions created in Step 2** (here, 4) (if refinement was done with 64 processes as in `run_visco.sh`, the simulation must also be run with `-np 64`):

```

mpirun -np 4 ./build/src/semsws \
  --config config_visco_sim.yaml

```

5.6 Visualization Scripts

Python visualization scripts are provided in `scripts/plot/`. All of them require `matplotlib` and `numpy`.

5.6.1 `plot_wavfield_2d.py` — 2D Wavefield Visualization

Visualizes 2D wavefield snapshots and material distributions output in GMT format, using `matplotlib`.

```

python3 scripts/plot/plot_wavfield_2d.py --results-dir ./results

```

The following files are generated in `results/figures/`:

- `wavfield_snapshots.png` — wavefield snapshots (all time steps)
- `material.png` — material distribution

See Figure 3.1 in Chapter 3 for a sample output.

5.6.2 plot_wavefield_3d.py — 3D Cross-Section Wavefield Visualization

Visualizes 3D cross-sections (cross_sections) in GMT format using matplotlib.

```
python3 scripts/plot/plot_wavefield_3d.py --results-dir ./results
```

A snapshot is generated for each cross-section (yz, xz, xy) specified in the config. Figure 5.2 shows a sample output of the xz cross-section ($y = 5000$ m) for the two-layer viscoelastic model.

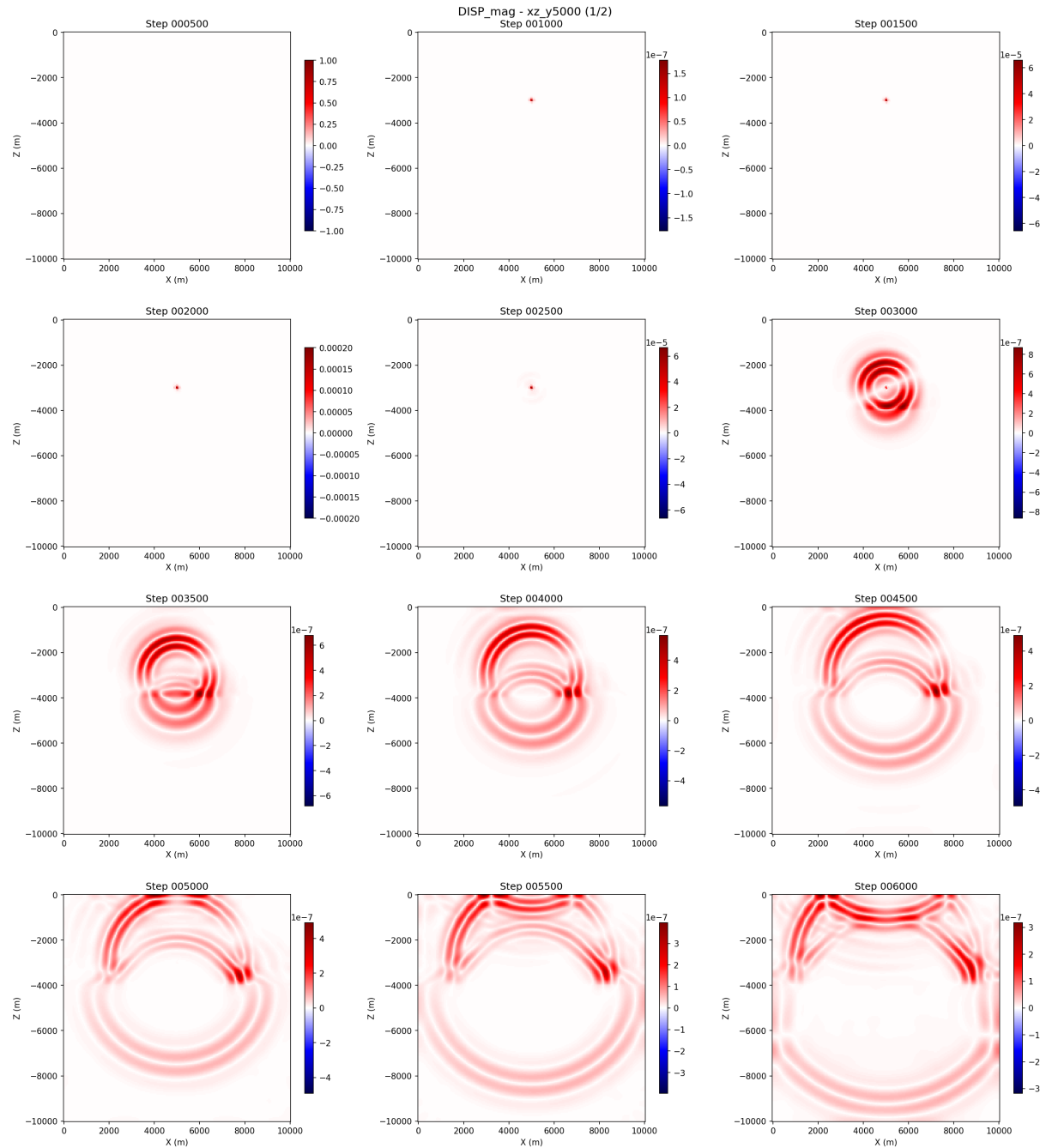


Figure 5.2: 3D cross-section snapshot from plot_wavefield_3d.py (xz cross-section, $y = 5000$ m). Displacement magnitude for the two-layer viscoelastic model. Output every 500 steps.

5.6.3 plot_3d_slices.py — PyVista 3D Visualization

Uses PyVista to visualize three cross-sections arranged in 3D space. The `pyvista` package is additionally required.

```
python3 scripts/plot/plot_3d_slices.py --results-dir ./results \
  --type wavefield --step 6000
```

Option	Description
<code>--results-dir</code>	Results directory
<code>--type</code>	<code>wavefield</code> / <code>material</code> / <code>both</code> (default)
<code>--step</code>	Time step number to display (for wavefield)

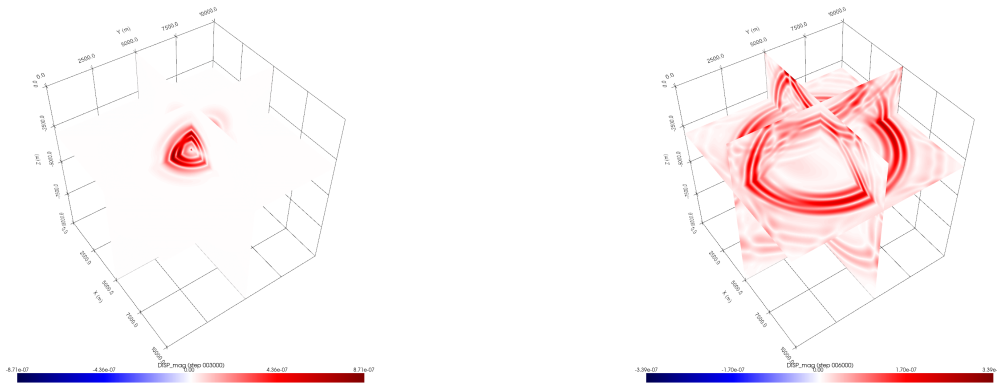


Figure 5.3: PyVista 3D visualization from `plot_3d_slices.py`. `yz`, `xz`, and `xy` cross-sections arranged in 3D space. Left: Step 3000, right: Step 6000. Displacement magnitude for the two-layer viscoelastic model.

5.6.4 plot_mesh_quality.py — Mesh Quality Histograms

Generates histogram figures from the CSV files output by `evaluate_mesh`.

```
# Output CSV with evaluate_mesh
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml \
  --histogram mesh_hist

# Generate histogram figures
python3 scripts/plot/plot_mesh_quality.py mesh_hist
python3 scripts/plot/plot_mesh_quality.py mesh_hist --output quality.png
```

Histograms for element size, CFL $\Delta t_{\max}$, PPW, aspect ratio, and the Jacobian determinant are combined into a single figure. The output resolution can be specified with `--dpi` (default 150).

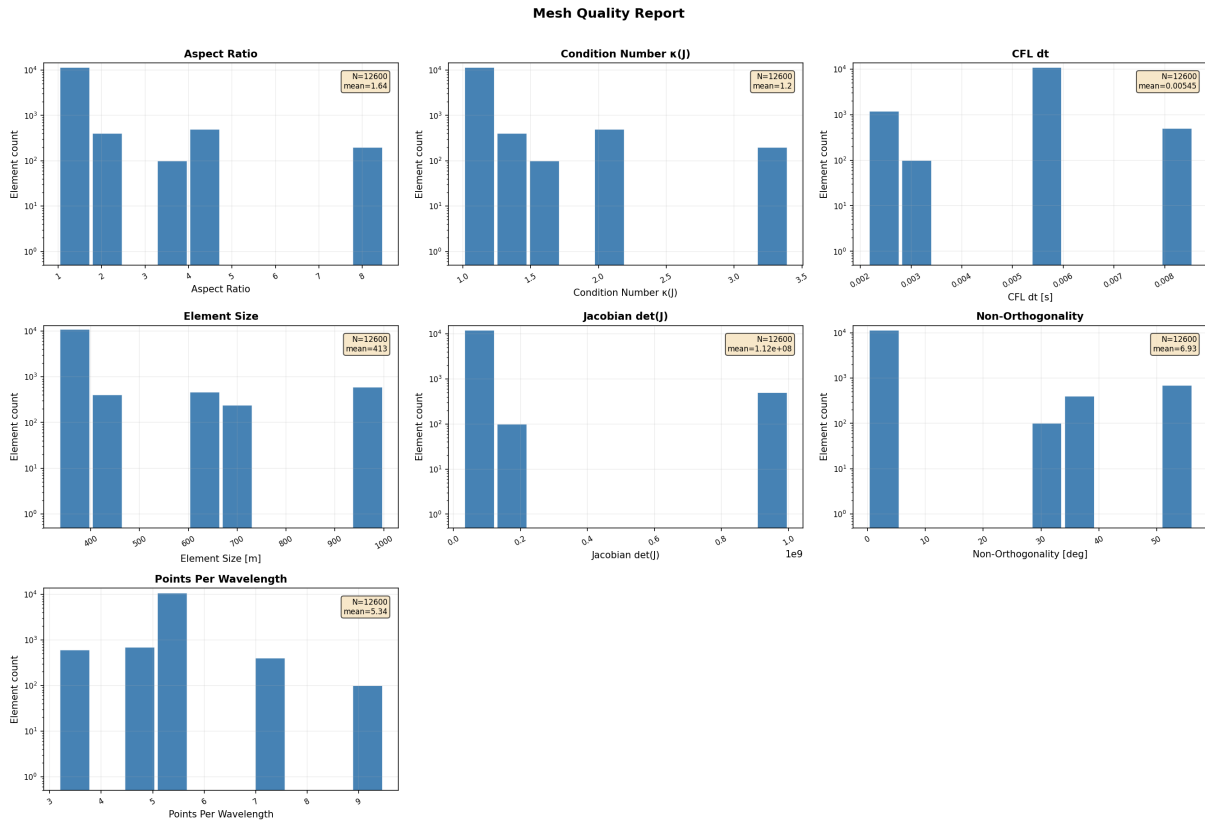


Figure 5.4: Mesh quality histograms (initial mesh before refinement). Output from `evaluate_mesh` visualized with `plot_mesh_quality.py`.

Appendix A

Site-Specific Build Configurations

This appendix summarizes manual build procedures specific to HPC sites, along with example specs for use with Spack. Refer to it together with §2.3 and §2.2 in the main text.

A.1 Earth Simulator (NVIDIA A100 GPU)

This section presents the procedure for JAMSTEC’s Earth Simulator GPU partition (NVIDIA A100). The following configuration was verified as of April 2025: NVIDIA HPC SDK 24.7, CUDA 12.0.0, hypr 2.31.0–2.33, METIS 5.1.0, HDF5 1.10.5.

A.1.1 Manual Build

```
# Load modules
module load HDF5/1.10.5
module load NetCDF4/all
module load NVIDIAHPCSDK/24.7/nvhpc-hpcx-cuda12
module load METIS/5.1.0
module load CUDA/12.0.0

# hypr
cd $HOME/program_gpu/hypr/src
./configure \
  --enable-single --enable-mixedint --enable-unified-memory \
  --with-cuda --enable-gpu-aware-mpi \
  CUDA_HOME=/opt/share/CUDA/12.0.0 HYPRE_CUDA_SM=80
make -j$(nproc)

# MFEM
cd $HOME/program_gpu/mfem
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=nvc++ \
  -DCMAKE_CUDA_COMPILER=nvcc \
  -DCMAKE_CUDA_ARCHITECTURES=80 \
  -DMFEM_USE_MPI=ON -DMFEM_USE_CUDA=ON \
```

```

-DMFEM_USE_HDF5=ON -DMFEM_USE_NETCDF=ON \
-DMFEM_PRECISION=single \
-DHYPRE_DIR=$HOME/program_gpu/hyprc/src/hyprc \
-DMETIS_DIR=/opt/share/METIS/5.1.0
cmake --build build -j$(nproc)

# SEMSWs
cd $HOME/program_gpu/SEMSWS
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=<install_dir> \
  -DCMAKE_CXX_COMPILER=nvc++ \
  -DCMAKE_CUDA_COMPILER=nvcc \
  -DCMAKE_CUDA_ARCHITECTURES=80 \
  -DSEM_USE_GPU_AWARE_MPI=ON \
  -DMFEM_DIR=$HOME/program_gpu/mfem/build \
  -DAdios2_DIR=$HOME/program_cpu/ADIOS2/build
cmake --build build -j
cmake --install build

```

A.1.2 Spack

On ES, external registration of the compiler, MPI, and CUDA is required, along with propagation flags specified in the spec. `spack-repo/site-config/es/` provides a load script and a `packages.yaml` template.

GPU env

```

export WORK=/S/data01/G3506/${USER}/program_test
git clone --depth=1 https://github.com/spack/spack.git $WORK/spack
cd $WORK && git clone https://github.com/SEMSWS/semsws.git && cd semsws

cp spack-repo/site-config/es/load_gpu.sh ~/load_gpu.sh
chmod +x ~/load_gpu.sh
source ~/load_gpu.sh

spack compiler find /usr/bin
spack repo add $WORK/semsws/spack-repo
spack env create semsws-gpu && spack env activate semsws-gpu
spack config edit          # Paste packages_gpu.yaml and the specs below

```

In the `spack.yaml` opened by `spack config edit`, paste the contents of `spack-repo/site-config/es/packages_gpu.yaml` and add the specs:

```

spack:
  repos:
    - /S/data01/G3506/<USER>/program_test/semsws/spack-repo
  packages:
    # Paste the contents of packages_gpu.yaml here
  specs:

```

```
- semsws@main +cuda cuda_arch=80 precision=single +gpu_aware_mpi
  cxxflags=="-noswitcherror" cflags=="-noswitcherror"
  ldlibs=="-lstdc++fs"
  ^hypre@2.33
  %nvhpc
concretizer:
  unify: true
config:
  build_jobs: 4
```

Then:

```
spack concretize -f
spack install --fail-fast 2>&1 | tee install_gpu.log
```

The first full build takes 50–90 minutes (nvcc compilation of the hypre CUDA kernels alone takes over 30 minutes).

CPU env Managed as a separate env from the GPU version. Because the NVIDIA HPC SDK and the system OpenMPI cannot be loaded at the same time, switch so that only one of `load_gpu.sh` and `load_cpu.sh` is sourced at a time:

```
spack env deactivate
source ~/load_cpu.sh
spack env create semsws-cpu && spack env activate semsws-cpu
spack config edit          # Paste packages_cpu.yaml, specs:
                           # - semsws@main precision=single ^hypre@2.33 %gcc
spack concretize -f
spack install --fail-fast 2>&1 | tee install_cpu.log
```

A.1.3 Subsequent Sessions

```
# GPU version
source ~/load_gpu.sh && spack env activate semsws-gpu && spack load semsws

# CPU version
source ~/load_cpu.sh && spack env activate semsws-cpu && spack load semsws
```

`source` is required because only one of the GPU or CPU system modules can be used at a time (activating the Spack env alone does not switch the module environment).

A.1.4 Example PJM Job Script

```
#!/bin/bash
#PJM -L rscgrp=gpu
#PJM -L node=1
#PJM -L elapse=01:00:00
#PJM -j
```

```

source ~/load_gpu.sh
spack env activate semsws-gpu
spack load semsws

# 8 ranks on 1 node (bound to each of the 8 A100s)
mpirun -np 8 -map-by ppr:8:node \
      -x PATH -x LD_LIBRARY_PATH \
      semsws --config config.yaml

```

For multi-node runs, always add `-x PATH -x LD_LIBRARY_PATH`. Because `spack load` sets `PATH/LD_LIBRARY_PATH` only in the shell on the head node, forgetting this causes `semsws: command not found` or `cannot open shared object file` for shared libraries on remote nodes.

A.2 LUMI (AMD MI250X GPU)

This section presents the procedure for CSC Finland’s LUMI supercomputer (LUMI-G partition). The following configuration was verified as of April 2026: ROCm 6.4.4, Cray PE 25.09 (CCE 20.0.0), hypre 3.1.0, METIS 5.1.0 (cpeCray-25.09-idx32-fp32), cray-hdf5 1.14.3.7.

A.2.1 Manual Build

```

# Load modules
module load LUMI/25.09 partition/G
module load METIS/5.1.0-cpeCray-25.09-idx32-fp32
module load cray-hdf5/1.14.3.7

# hypre
cd $FD/program/hypre/src
./configure \
  --enable-single --enable-mixedint --enable-unified-memory \
  --with-hip --enable-gpu-aware-mpi \
  CC=hipcc CXX=hipcc \
  LDFLAGS="-L/opt/cray/pe/mpich/9.0.1/gtl/lib" \
  LIBS="-lmpi_gtl_hsa"
make -j$(nproc)

# MFEM
cd $FD/program/mfem
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=hipcc \
  -DMFEM_USE_MPI=ON -DMFEM_USE_HIP=ON \
  -DAMDGPU_TARGETS=gfx90a \
  -DMFEM_USE_HDF5=ON -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=$FD/program/hypre/src/hypre \
  -DHDF5_INCLUDE_DIRS=/opt/cray/pe/hdf5/1.14.3.7/cray/20.0/include \

```

```
-DHDF5_LIBRARIES="/opt/cray/pe/hdf5/1.14.3.7/cray/20.0/lib/libhdf5_cpp.so
;\
/opt/cray/pe/hdf5/1.14.3.7/cray/20.0/lib/libhdf5.so"
cmake --build build -j$(nproc)

# SEMSW
cd $FD/program/SEMSW
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=<install_dir> \
  -DCMAKE_CXX_COMPILER=hipcc \
  -DAMDGPU_TARGETS=gfx90a \
  -DSEM_USE_GPU_AWARE_MPI=ON \
  -DMFEM_DIR=$FD/program/mfem/build \
  -DAdios2_DIR=$FD/program/ADIOS2/build
cmake --build build -j
cmake --install build
```

Use `srund` to submit MPI jobs.

Development Team

- Kota Mukumoto — Barcelona Center for Subsurface Imaging, Institut de Ciències del Mar (CSIC), Barcelona, Spain
- Yann Capdeville — Nantes Université, Univ Angers, Le Mans Université, CNRS, Laboratoire de Planétologie et Géosciences, Nantes, France
- Kazuya Shiraishi — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Ryoichiro Agata — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Gou Fujie — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Thomas Bodin — Barcelona Center for Subsurface Imaging, Institut de Ciències del Mar (CSIC), Barcelona, Spain